	Katedra Grafiki Wizji Komputerowej i Systemów Cyfrowych **Raport końcowy JA**		Politechnika Śląska
Rok akademicki	**Rodzaj studiów:**	**Dzień tygodnia i godzina zajęć:**	**Numer sekcji:**
2025/2026	**SSI KTW**	**Środa 14:30 – 16:00**	**3**
Imię: **Nazwisko:**	**Błażej** **Jamrozik**	**Prowadzący** (inicjały**)**:	**KH**
Temat projektu			
ASCII ART Sztuka tworzona za pomocą znaków kodu ASCII			

Spis treści

1 Spis rysunków 4
2 Spis tabel 5
3 Wprowadzenie 6
4 Szczegółowy opis interfejsu użytkownika, parametrów wejściowych i działania aplikacji 6
4.1 Sekcje interfejsu użytkownika 7
4.1.1 Sekcja wizualizacji 7
4.1.2 Sekcja sterowania i konfiguracji 7
4.1.3 Sekcja informacyjna i monitorowania 7
4.2 Tryb pracy z pojedynczym obrazem 7
4.2.1 Wybór i wczytywanie obrazu 7
4.2.2 Proces konwersji pojedynczego obrazu 8
4.2.3 Wyniki i komunikaty dla konwersji pojedynczego obrazu 8
4.3 Tryb pracy z całym folderem obrazów 9
4.3.1 Wybór i inicjalizacja folderu 9
4.3.2 Proces konwersji folderu 10
4.3.3 Wyniki i raportowanie dla folderu 10
4.4 Mechanizmy obsługi błędów i powiadomień 11
4.5 Zamykanie 12
4.6 System zapisu wyników i pomiarów 12
4.7 Diagramy przypadków użycia 13
5 Opis części asemblerowej 14
5.1 Ogólny podział kodu 14
5.2 Przykładowy fragment kodu asemblerowego 15
6 Wyniki prędkości wykonania algorytmu 17
6.1 Analiza czasów wykonania całego programu 17
6.1.1 Zestaw danych S (5 obrazów) 17
6.1.2 Zestaw danych M (20 obrazów) 19
6.1.3 Zestaw danych L (50 obrazów) 21
6.1.4 Porównanie zbiorowe wszystkich zestawów danych 23

6.2 Analiza czasów wykonania bibliotek DLL ... 25
6.2.1 Zestaw danych S (5 obrazów) ... 25
6.2.2 Zestaw danych M (20 obrazów) ... 27
6.2.3 Zestaw danych M (50 obrazów) ... 29
6.2.4 Porównanie zbiorowe wszystkich zestawów danych ... 31
7 Uruchamianie, testowanie i debugowanie ... 33
7.1 Uruchamianie aplikacji ... 33
7.2 Testowanie i walidacji ... 33
7.2.1 Testowanie pojedynczych obrazów ... 33
7.2.2 Testowanie folderów obrazów ... 34
7.3 Debugowanie ... 34
7.3.1 Debugowanie kodu w C++ ... 34
7.3.2 Debugowanie kodu asemblerowego ... 34
8 Podsumowanie ... 35
8.1 Wnioski dotyczące działania całego programu ... 35
8.2 Wnioski dotyczące wielowątkowości ... 35
8.2.1 Wielowątkowość a czas wykonania funkcji DLL ... 35
8.2.2 Rzeczywiste źródła przyspieszenia ... 36
8.2.3 Optymalna liczba wątków i granice skalowalności ... 36
8.3 Wnioski dotyczące różnic między wersjami bibliotek DLL ... 36

1 Spis rysunków

Rys. 1. Początkowe okno aplikacji 6
Rys. 2. Okno wyboru typu danych do konwersji 8
Rys. 3. Okno aplikacji w trakcie konwersji pojedynczego obrazu 8
Rys. 4. Okno informujące o zapisie pliku w dedykowanym folderze 9
Rys. 5. Okno aplikacji po konwersji pojedynczego obrazu 9
Rys. 6. Okno aplikacji przed konwersją całego folderu obrazów 10
Rys. 7. Okno aplikacji w trakcie konwersji całego folderu obrazów 10
Rys. 8. Okno informujące o zapisie obrazów po konwersji w dedykowanym folderze 11
Rys. 9. Okno aplikacji po konwersji całego folderu obrazów 11
Rys. 10. Okno błędu 12
Rys. 11. Uproszczony diagram przypadków użycia 13
Rys. 12. Szczegółowy diagram przypadków użycia 14
Rys. 13. Wykres zależności średnich czasów wykonania całego programu dla kolekcji danych S od liczby wątków z podziałem na wersje DLL 19
Rys. 14. Wykres zależności średnich czasów wykonania całego programu dla kolekcji danych M od liczby wątków z podziałem na wersje DLL 21
Rys. 15. Wykres zależności średnich czasów wykonania całego programu dla kolekcji danych L od liczby wątków z podziałem na wersje DLL 23
Rys. 16. Wykres zależności średnich czasów wykonania całego programu dla kolekcji danych S, M, L od liczby wątków z podziałem na wersje DLL 24
Rys. 17. Wykres zależności średnich czasów wykonania DLL w różnych wersjach dla kolekcji danych S od liczby wątków 26
Rys. 18. Wykres zależności średnich czasów wykonania DLL w różnych wersjach dla kolekcji danych M od liczby wątków 28
Rys. 19. Wykres zależności średnich czasów wykonania DLL w różnych wersjach dla kolekcji danych L od liczby wątków 30
Rys. 20. Wykres zależności średnich czasów wykonania DLL w wersji ASM oraz CPP z optymalizacją O2 dla kolekcji danych S, M, L od liczby wątków 31
Rys. 21. Wykres zależności średnich czasów wykonania DLL w wersji CPP dla kolekcji danych S, M, L od liczby wątków 32

2 Spis tabel

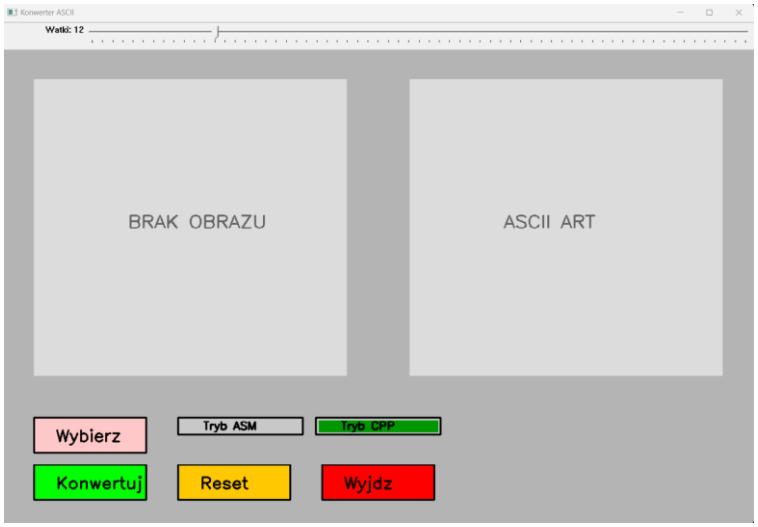
Tab. 1. Zestawienie czasów wykonania całego programu dla kolekcji danych S z DLL w wersji ASM 17
Tab. 2. Zestawienie czasów wykonania całego programu dla kolekcji danych S z DLL w wersji CPP. 18
Tab. 3. Zestawienie czasów wykonania całego programu dla kolekcji danych S z DLL w wersji CPP z optymalizacją O2 18
Tab. 4. Zestawienie czasów wykonania całego programu dla kolekcji danych M z DLL w wersji ASM 19
Tab. 5. Zestawienie czasów wykonania całego programu dla kolekcji danych M z DLL w wersji CPP 20
Tab. 6. Zestawienie czasów wykonania całego programu dla kolekcji danych M z DLL w wersji CPP z optymalizacją O2 20
Tab. 7. Zestawienie czasów wykonania całego programu dla kolekcji danych L z DLL w wersji ASM 21
Tab. 8. Zestawienie czasów wykonania całego programu dla kolekcji danych L z DLL w wersji CPP. 22
Tab. 9. Zestawienie czasów wykonania całego programu dla kolekcji danych L z DLL w wersji CPP z optymalizacją O2 22
Tab. 10. Zestawienie średnich czasów wykonania całego programu dla kolekcji danych S, M, L z podziałem na wersje DLL 23
Tab. 11. Zestawienie czasów wykonania DLL w wersji ASM dla kolekcji danych S 25
Tab. 12. Zestawienie czasów wykonania DLL w wersji CPP dla kolekcji danych S 25
Tab. 13. Zestawienie czasów wykonania DLL w wersji CPP z optymalizacją O2 dla kolekcji danych S 26
Tab. 14. Zestawienie czasów wykonania DLL w wersji ASM dla kolekcji danych M 27
Tab. 15. Zestawienie czasów wykonania DLL w wersji CPP dla kolekcji danych M 27
Tab. 16. Zestawienie czasów wykonania DLL w wersji CPP z optymalizacją O2 dla kolekcji danych M 28
Tab. 17. Zestawienie czasów wykonania DLL w wersji ASM dla kolekcji danych L 29
Tab. 18. Zestawienie czasów wykonania DLL w wersji CPP dla kolekcji danych L 29
Tab. 19. Zestawienie czasów wykonania DLL w wersji CPP z optymalizacją O2 dla kolekcji danych L 30
Tab. 20. Zestawienie średnich czasów wykonania DLL w wersji ASM oraz CPP z optymalizacją O2 dla kolekcji danych S,M,L 31
Tab. 21. Zestawienie średnich czasów wykonania DLL w wersji CPP dla kolekcji danych S, M, L 32

3 Wprowadzenie

Projekt realizuje konwerter obrazów do postaci ASCII Art. Algorytm konwersji opiera się na transformacji wartości jasności pikseli w skali szarości na odpowiadające im znaki ASCII z predefiniowanej palety 64 znaków, uporządkowanych według rosnącej jasności wizualnej – od najciemniejszych symboli o wysokiej gęstości graficznej takich jak "$" i "@" poprzez znaki o średniej intensywności jak litery i cyfry, aż do najjaśniejszych reprezentacji w postaci znaków interpunkcyjnych i spacji. Proces mapowania realizowany jest poprzez zastosowanie arytmetyki stałoprzecinkowej (fixed-point arithmetic), gdzie wartość jasności piksela z zakresu 0-255 jest przekształcana na indeks w tablicy znaków przez operację (jasność × 63) >> 8, co efektywnie dzieli zakres wartości jasności na 64 równe przedziały odpowiadające poszczególnym znakom palety ASCII. Kluczowym aspektem projektu jest implementacja dwóch wersji funkcjonalnie równoważnych algorytmu konwersji – zoptymalizowanej wersji asemblerowej wykorzystującej rozszerzenia SIMD SSE4.1 umożliwiające równoległe przetwarzanie 16 pikseli jednocześnie oraz referencyjnej implementacji w języku C++ dostępnej zarówno bez optymalizacji jak i z flagą kompilatora O2.

4 Szczegółowy opis interfejsu użytkownika, parametrów wejściowych i działania aplikacji

Interfejs użytkownika został zaprojektowany jako aplikacja okienkowa wykorzystująca bibliotekę OpenCV do renderowania graficznego interfejsu. Główne okno aplikacji o wymiarach 1200×800 pikseli organizuje funkcjonalność w logicznie wydzielonych sekcjach, z których każda pełni określoną rolę w procesie przetwarzania.



Rys. 1. Początkowe okno aplikacji

4.1 Sekcje interfejsu użytkownika

4.1.1 Sekcja wizualizacji

Sekcja wizualizacji zajmuje górną część interfejsu, dzieląc przestrzeń na dwa symetryczne panele o wymiarach 500×500 pikseli każdy. Lewy panel służy do prezentacji oryginalnego obrazu wejściowego. W stanie początkowym, gdy żaden obraz nie został załadowany, panel wyświetla jednolite szare tło z napisem "BRAK OBRAZU". Prawy panel odpowiada za wyświetlanie wyników konwersji ASCII Art. W stanie spoczynku prezentuje szare tło z napisem "ASCII ART", który po wykonaniu konwersji zastępowany jest renderowanym obrazem stworzonym ze znaków ASCII.

4.1.2 Sekcja sterowania i konfiguracji

Sekcja sterowania, zajmująca dolny obszar interfejsu, organizuje przyciski funkcjonalne w dwóch poziomych rzędach. W górnym rzędzie znajduje się przycisk "Wybierz" z jasnoróżowym wypełnieniem, który inicjuje systemowy dialog wyboru trybu importu. Obok przycisku wyboru umieszczono dwa interaktywne checkboxy "Tryb ASM" i "Tryb CPP", które umożliwiają przełączanie między implementacjami algorytmu konwersji. Aktywny tryb wizualnie wyróżniany jest poprzez zielone wypełnienie wewnątrz checkboxa.

Dolny rząd zawiera trzy przyciski operacyjne o identycznych wymiarach: "Konwertuj" z zielonym wypełnieniem uruchamiający proces transformacji, "Reset" z pomarańczowym wypełnieniem przywracający stan początkowy aplikacji oraz "Wyjdz" z czerwonym wypełnieniem bezpiecznie zamykający program.

Na górze interfejsu znajduje się suwak "Watki" umożliwiający regulację liczby wątków przetwarzania w zakresie od 1 do 64. Domyślna wartość jest automatycznie ustawiana na liczbę rdzeni logicznych procesora wykrywaną przez *std::thread::hardware_concurrency()*.

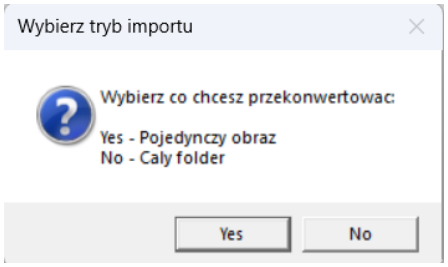
4.1.3 Sekcja informacyjna i monitorowania

Pomiędzy panelami wizualizacji a sterowaniem znajduje się obszar informacyjny dostarczający użytkownikowi kluczowych danych o stanie systemu. Komunikaty nad przyciskami wyświetlane są w dwóch kolorach: czerwonym dla informacji o stanie procesu (postęp konwersji, wybór folderu) i czarnym dla wyników wydajnościowych po zakończeniu przetwarzania.

4.2 Tryb pracy z pojedynczym obrazem

4.2.1 Wybór i wczytywanie obrazu

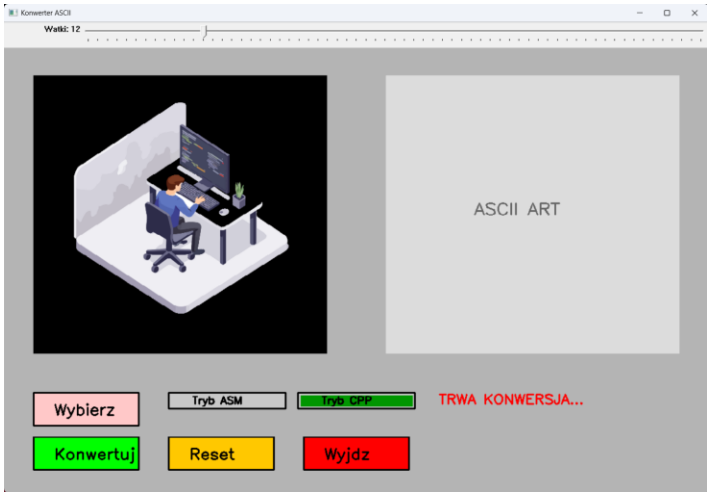
Po kliknięciu przycisku "Wybierz", użytkownik otrzymuje okno dialogowe z pytaniem o tryb importu. Wybór opcji "Yes - Pojedynczy obraz" otwiera standardowe okno wyboru pliku z predefiniowanymi filtrami formatów graficznych (PNG, JPG, JPEG, BMP). Po wybraniu pliku, system automatycznie wczytuje obraz i wyświetla go w lewym panelu wizualizacji z zachowaniem oryginalnych proporcji. W tym trybie prawy panel pozostaje pusty z napisem "ASCII ART" do momentu wykonania konwersji.



Rys. 2. Okno wyboru typu danych do konwersji

4.2.2 Proces konwersji pojedynczego obrazu

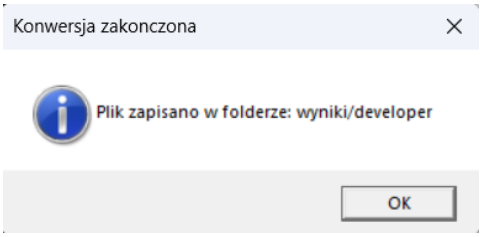
Kliknięcie przycisku "Konwertuj" inicjuje proces transformacji dla załadowanego obrazu. Podczas konwersji pojawia się czerwony komunikat "TRWA KONWERSJA...". System wykonuje wieloetapowe przetwarzanie: konwersję do skali szarości, skalowanie do docelowej szerokości 400 znaków z dynamicznym obliczaniem wysokości oraz właściwą konwersję ASCII przy użyciu wybranej biblioteki DLL.



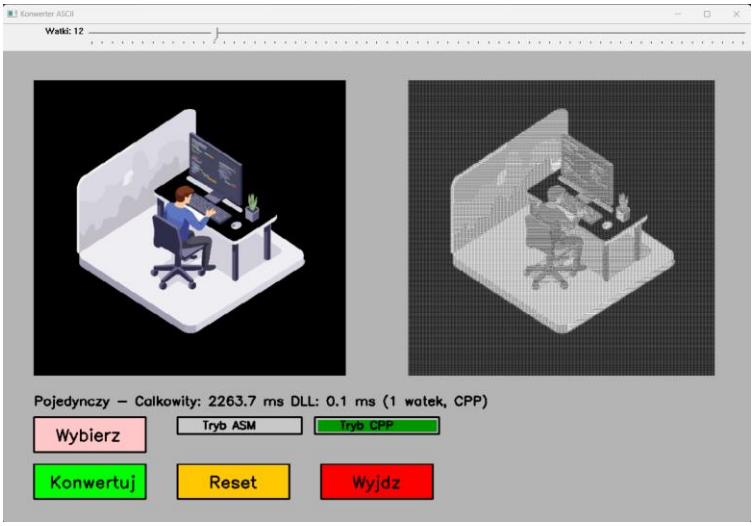
Rys. 3. Okno aplikacji w trakcie konwersji pojedynczego obrazu

4.2.3 Wyniki i komunikaty dla konwersji pojedynczego obrazu

Po pomyślnym zakończeniu konwersji, prawy panel wizualizacji wypełnia się graficzną reprezentacją ASCII Art, a nad przyciskami pojawia się czarny komunikat w formacie "Pojedynczy - Calkowity: X ms DLL: Y ms (Z watkow, TRYB)", gdzie X to całkowity czas działania aplikacji, Y to czas działania algorytmu ze stworzonej biblioteki dll, Z to liczba wątków użyta podczas przetwarzania, a TRYB wskazuje zastosowaną implementację (ASM lub CPP). Jednocześnie system wyświetla okno dialogowe z informacją "Plik zapisano w folderze: wyniki/[nazwa_obrazu]".



Rys. 4. Okno informujące o zapisie pliku w dedykowanym folderze

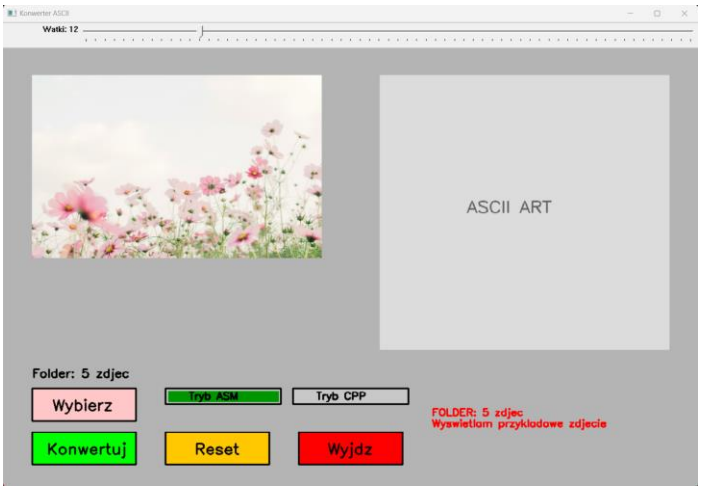


Rys. 5. Okno aplikacji po konwersji pojedynczego obrazu

4.3 Tryb pracy z całym folderem obrazów

4.3.1 Wybór i inicjalizacja folderu

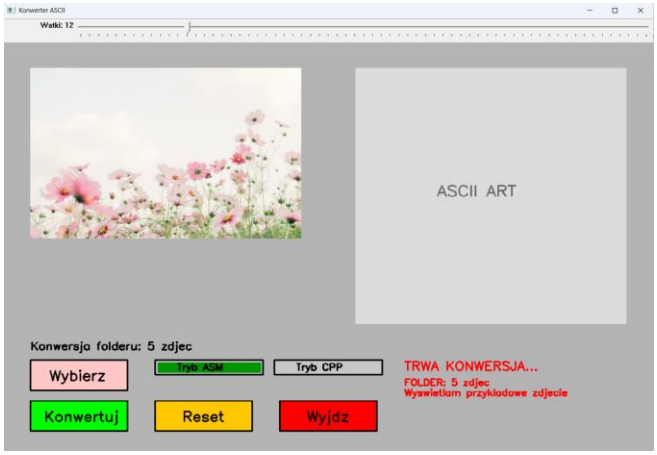
Wybór opcji "No - Cały folder" w oknie dialogowym trybu importu (Rys. 2. Okno wyboru typu danych do konwersji) otwiera systemowe okno wyboru katalogu. System automatycznie przeszukuje wybrany folder i wszystkie jego podfoldery w poszukiwaniu plików graficznych (PNG, JPG, JPEG, BMP). Po pomyślnym znalezieniu obrazów, system wyświetla w lewym panelu przykładowy obraz z folderu, a w prawym dolnym rogu pojawiają się dwa czerwone komunikaty: "FOLDER: X zdjec" oraz "Wyswietlam przykladowe zdjecie", gdzie X to liczba znalezionych plików obrazów. Dodatkowo nad przyciskiem "Wybierz" wyświetla się komunikat z liczbą zdjęć z folderu.



Rys. 6. Okno aplikacji przed konwersją całego folderu obrazów

4.3.2 Proces konwersji folderu

Kliknięcie przycisku "Konwertuj" uruchamia się równoległe przetwarzanie wszystkich obrazów z wykorzystaniem puli wątków. Podczas konwersji w interfejsie wyświetlany jest statyczny komunikat "TRWA KONWERSJA...". System przetwarza obrazy równolegle, dzieląc pulę plików między dostępne wątki robocze, gdzie każdy wątek niezależnie konwertuje przypisane mu obrazy..

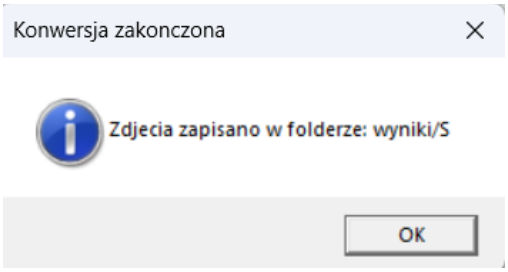


Rys. 7. Okno aplikacji w trakcie konwersji całego folderu obrazów

4.3.3 Wyniki i raportowanie dla folderu

Po zakończeniu przetwarzania wszystkich obrazów, system wyświetla okno dialogowe z informacją "Zdjecia zapisano w folderze: wyniki/[nazwa_folderu]". Nad przyciskami pojawia się czarny komunikat podsumowujący w formacie "Folder - Calkowity: X ms DLL (suma): Y ms (Z zdjec, W watkow)", gdzie X

to całkowity czas przetwarzania folderu, Y to łączny czas pracy bibliotek DLL dla wszystkich obrazów, Z to liczba przetworzonych plików, a W to liczba wykorzystanych wątków. System automatycznie generuje zbiorczy raport w pliku CSV zawierający szczegółowe dane czasowe dla całego procesu konwersji folderu.



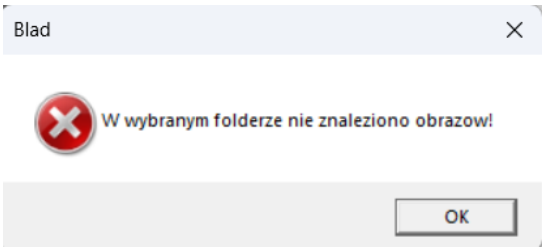
Rys. 8. Okno informujące o zapisie obrazów po konwersji w dedykowanym folderze

Rys. 9. Okno aplikacji po konwersji całego folderu obrazów

4.4 Mechanizmy obsługi błędów i powiadomień

System implementuje zaawansowane mechanizmy obsługi błędów i powiadomień dla obu trybów pracy. W przypadku nieznalezienia obrazów w wybranym folderze, użytkownik otrzymuje okno dialogowe z komunikatem błędu. Dla pojedynczego obrazu, w przypadku problemów z wczytaniem pliku, system wyświetla odpowiedni komunikat w konsoli.

Po pomyślnym zakończeniu konwersji w obu trybach system wyświetla okno informacyjne z potwierdzeniem zapisania wyników. Dla trybu pojedynczego obrazu informacja dotyczy zapisania pojedynczego pliku, podczas gdy dla trybu folderu - wszystkich przetworzonych obrazów.



Rys. 10. Okno błędu

4.5 Zamykanie

Aplikacja może być bezpiecznie zamykana na trzy sposoby: przez przycisk "Wyjdz", przez naciśnięcie klawisza ESC, lub przez standardowy przycisk X w ramce okna. Każda z tych metod zapewnia poprawne zwolnienie wszystkich zasobów i zamknięcie okna bez generowania wyjątków.

4.6 System zapisu wyników i pomiarów

System zapisu wyników automatycznie generuje pliki wyjściowe w dedykowanej strukturze katalogów. Folder główny "wyniki" jest tworzony automatycznie w bieżącym katalogu wykonania aplikacji.

Struktura katalogów:

- Dla pojedynczego obrazu: wyniki/[nazwa_obrazu]/
- Dla folderu: wyniki/[nazwa_folderu_wejscie]/[nazwa_obrazu]/

Pliki wynikowe w każdym podfolderze obrazu:

- [nazwa_obrazu]_ASCII_[TRYB].txt - czysta postać ASCII Art
- [nazwa_obrazu]_ASCII_[TRYB].png - renderowana reprezentacja wizualna

Dane pomiarowe zapisywane są w plikach CSV:

Dla pojedynczych obrazów ([TRYB]_czasy.csv w podfolderze nadrzędnym):

- "Watki" - liczba wątków użyta podczas konwersji (zawsze 1 dla pojedynczego obrazu)
- "CzasCalkowity[ms]" - całkowity czas przetwarzania obrazu
- "CzasDLL[ms]" - łączny czas wykonania funkcji z DLL

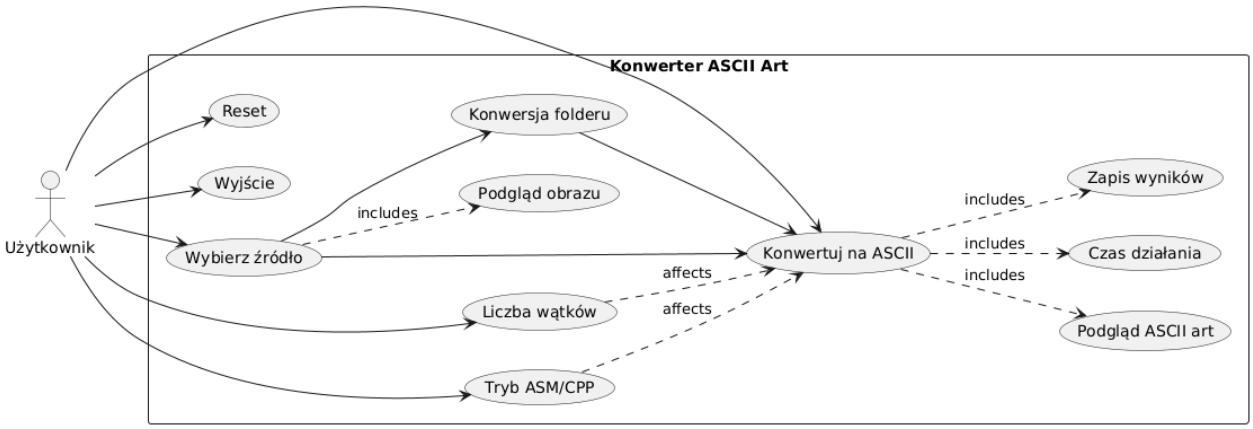
- "NazwaPliku" - nazwa przetworzonego pliku źródłowego

Dla konwersji folderów (raport_konwersji_folderow_[TRYB].csv w folderze głównym "wyniki"):

- "Data" - data i czas wykonania konwersji całego folderu
- "Watki" - liczba wątków użyta do równoległego przetwarzania wielu obrazów
- "CzasCalkowity[ms]" - całkowity czas przetworzenia całego folderu
- "CzasDLL[ms]" - suma wszystkich czasów DLL z poszczególnych obrazów w folderze
- "LiczbaPlikow" - całkowita liczba przetworzonych obrazów
- "NazwaFolderu" - nazwa folderu źródłowego

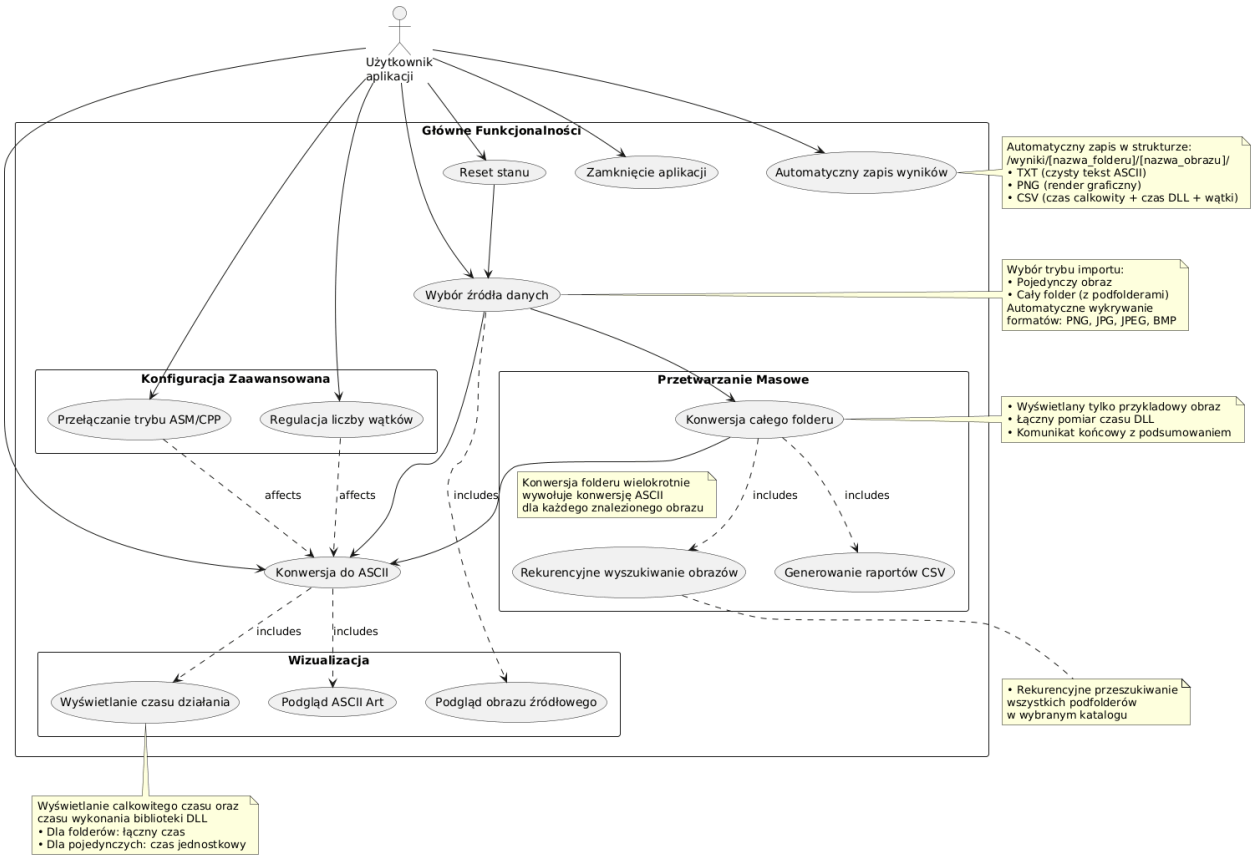
4.7 Diagramy przypadków użycia

Diagram przypadków użycia przedstawia główne funkcjonalności systemu konwertera ASCII Art z perspektywy użytkownika końcowego, pokazując wszystkie kluczowe interakcje.



Rys. 11. Uproszczony diagram przypadków użycia

Drugi, bardziej szczegółowy diagram rozszerza tę wizję o podział na grupy funkcjonalności - podstawowe operacje, zaawansowaną konfigurację, moduł wizualizacji, przetwarzanie masowe - oraz uwidacznia relacje i zależności między poszczególnymi przypadkami użycia.



Rys. 12. Szczegółowy diagram przypadków użycia

5 Opis części asemblerowej

W ramach projektu konwertera obrazów do ASCII Art kluczowym elementem optymalizacyjnym jest procedura napisana w asemblerze x64, która odpowiada za efektywną konwersję wartości jasności pikseli na odpowiadające im znaki ASCII. Plik *ascii_core.asm* zawiera zoptymalizowaną implementację wykorzystującą rozszerzenia SIMD (SSE4.1) w celu przetwarzania 16 pikseli jednocześnie, co znacząco przyspiesza działanie algorytmu. Główna koncepcja polega na zastosowaniu arytmetyki stałoprzecinkowej do mapowania wartości jasności z zakresu 0–255 na indeksy 0–63, które następnie są tłumaczone na znaki z predefiniowanej palety ASCII uporządkowanej według rosnącej jasności.

5.1 Ogólny podział kodu

Plik rozpoczyna się od sekcji danych, gdzie zdefiniowane są stałe niezbędne do działania algorytmu. Tablica ZNAKI zawiera 64 znaki ASCII uporządkowane od najciemniejszego do najjaśniejszego. Stała *fixed_scale* przechowuje wartość 63 powtórzoną 8 razy w formacie słów 16-bitowych, co jest wykorzystywane w obliczeniach fixed-point. Bufor *tmp_idx* służy jako pamięć pomocnicza do przechowywania tymczasowych indeksów podczas przetwarzania wektorowego.

Funkcja *get_required_buffer_size* oblicza minimalny rozmiar bufora wyjściowego potrzebnego do przechowania wyniku konwersji. Algorytm uwzględnia nie tylko znaki odpowiadające pikselom, ale również znaki nowej linii oraz znak końca bufora. Walidacja parametrów wejściowych zapewnia, że szerokość i wysokość obrazu są wartościami dodatnimi, a obliczony rozmiar bufora nie przekracza maksymalnej wartości 32-bitowej.

Funkcja *jasnoscNaASCII* implementuje skalarną konwersję pojedynczej wartości jasności na znak ASCII przy użyciu arytmetyki stałoprzecinkowej. Wartość jasności jest mnożona przez 63 i dzielona przez 256 poprzez przesunięcie bitowe, co efektywnie mapuje ją na zakres 0-63. Mechanizm clampowania zapewnia, że wynik zawsze znajduje się w poprawnym zakresie, a następnie na podstawie obliczonego indeksu wybierany jest odpowiedni znak z tablicy *ZNAKI*.

Główna funkcja *konwertuj_ascii_core* stanowi serce całego systemu konwersji. Po wstępnej walidacji parametrów wejściowych, procedura inicjalizuje stałe SSE i przechodzi do przetwarzania wierszy obrazu. Dla każdego wiersza obliczane są wskaźniki do danych źródłowych i docelowych, a następnie przetwarzanie odbywa się w dwóch etapach: wektorowym dla bloków po 16 pikseli i skalarnym dla pozostałych pikseli.

5.2 Przykładowy fragment kodu asemblerowego

```
; ==============================================================================
; PĘTLA WEKTOROWA (SSE) - FIXED-POINT
; ==============================================================================
pixel_loop_vector:
   cmp  r11d, 16         ; sprawdza czy pozostało >= 16 pikseli
   jl   pixel_loop_scalar ; jeśli mniej, przechodzi do przetwarzania skalarnego

   ; --- Załadowanie 16 pikseli do rejestru SSE ---
   movdqu xmm0, XMMWORD PTR [r8] ; ładuje 16 bajtów jasności do xmm0
   lea    rax, tmp_idx    ; rax = adres bufora pomocniczego

   ; --- Rozszerzenie bajtów 0-7 do words i przetworzenie ---
   pmovzxbw xmm1, xmm0    ; rozszerza bajty 0-7 do 8 words (16-bit)
   pmullw xmm1, xmm7      ; mnoży przez 63 (fixed-point)
   psrlw xmm1, 8          ; dzieli przez 256 (>> 8)

   ; --- Rozszerzenie bajtów 8-15 do words i przetworzenie ---
   psrldq xmm0, 8         ; przesuwa o 8 bajtów w prawo (bajty 8-15)
   pmovzxbw xmm2, xmm0    ; rozszerza bajty 8-15 do 8 words (16-bit)
   pmullw xmm2, xmm7      ; mnoży przez 63 (fixed-point)
   psrlw xmm2, 8          ; dzieli przez 256 (>> 8)

   ; --- Spakowanie wyników z powrotem do bajtów ---
   packuswb xmm1, xmm2    ; spakuje 16 words do 16 bajtów
   movdqu XMMWORD PTR [rax], xmm1 ; zapisuje wszystkie 16 indeksów do bufora

   ; --- Mapowanie indeksów na znaki ASCII ---
   push r13               ; zapisuje r13 (rozmiar bufora) na stosie
   push r14               ; zapisuje r14 (start_row/end_row) na stosie
   lea  r13, ZNAKI        ; r13 = adres tablicy znaków ASCII
   xor  r14, r14          ; r14 = licznik pętli (0-15)
```

```
lookup_loop16:
   cmp  r14, 16          ; sprawdza czy przetworzono 16 znaków
   jge  lookup_done      ; jeśli tak, kończy pętlę

   movzx ecx, BYTE PTR [rax + r14] ; odczytuje indeks z bufora tymczasowego
   mov   dl, BYTE PTR [r13 + rcx]  ; pobiera znak ASCII na podstawie indeksu
   mov   BYTE PTR [r9 + r14], dl   ; zapisuje znak do bufora wyjściowego

   inc   r14             ; zwiększa licznik
   jmp   lookup_loop16   ; kontynuuje pętlę

lookup_done:
   pop  r14              ; odczytuje poprzednią wartość r14
   pop  r13              ; odczytuje poprzednią wartość r13

   ; --- Aktualizacja wskaźników i liczników ---
   add  r8, 16           ; przesuwa wskaźnik źródłowy o 16 bajtów
   add  r9, 16           ; przesuwa wskaźnik docelowy o 16 bajtów
   sub  r11d, 16         ; zmniejsza licznik kolumn o 16
   jmp  pixel_loop_vector ; kontynuuje pętlę wektorową
```

W sekcji przetwarzania wektorowego realizowane jest jednoczesne przetwarzanie 16 pikseli. Fragment kodu zaczyna się od sprawdzenia, czy w bieżącym wierszu pozostało co najmniej 16 pikseli do przetworzenia. Jeśli tak, następuje załadowanie 16 kolejnych bajtów (reprezentujących jasności) do rejestru SSE xmm0 za pomocą instrukcji *movdqu*. Następnie adres bufora pomocniczego *tmp_idx* jest ładowany do rejestru RAX. Kolejnym krokiem jest rozszerzenie pierwszych 8 bajtów z xmm0 do 16-bitowych słów w rejestrze xmm1 przy użyciu *pmovzxbw*, co umożliwia dalsze operacje arytmetyczne. Wartości te są mnożone przez stałą 63 za pomocą *pmullw*, a następnie przesuwane bitowo w prawo o 8 pozycji (*psrlw*), co efektywnie dzieli je przez 256 i zwraca wartości z zakresu 0-63. Analogiczna operacja jest wykonywana dla kolejnych 8 bajtów (bajty 8-15) po przesunięciu zawartości xmm0 (*psrldq xmm0, 8*) i rozszerzeniu do xmm2. Otrzymane 16-bitowe wyniki są następnie pakowane z powrotem do bajtów w rejestrze xmm1 instrukcją *packuswb* i zapisywane w buforze tymczasowym.

Po uzyskaniu 16 indeksów w buforze *tmp_idx* procedura przechodzi do etapu mapowania tych indeksów na odpowiadające im znaki ASCII. W tym celu zapisywane są oryginalne wartości rejestrów R13 i R14 na stosie, a następnie ładowany jest adres tablicy znaków *ZNAKI* do R13. W pętli *lookup_loop16* dla każdego z 16 indeksów odczytywany jest odpowiedni znak z tablicy *ZNAKI* i zapisywany w buforze wyjściowym. Po zakończeniu pętli przywracane są oryginalne wartości rejestrów, a wskaźniki źródłowy i docelowy oraz licznik pozostałych pikseli są aktualizowane, co umożliwia kontynuowanie przetwarzania w pętli wektorowej.

Dla pozostałych pikseli (mniej niż 16 w bieżącym wierszu) algorytm wykorzystuje przetwarzanie skalarne, wywołując procedurę *jasnoscNaASCII* dla każdego pojedynczego piksela. Na koniec każdego wiersza dodawany jest znak nowej linii, co zapewnia poprawną strukturę tekstowego wyniku.

Mechanizm obsługi błędów zapewnia odpowiednie reagowanie na nieprawidłowe parametry wejściowe, takie: jak wskaźniki NULL, nieprawidłowe rozmiary obrazu czy błędny zakres wierszy do

przetworzenia. Każdy błąd jest sygnalizowany odpowiednim kodem powrotu, co umożliwia poprawne zarządzanie błędami.

6 Wyniki prędkości wykonania algorytmu

Analiza wydajności konwertera ASCII Art obejmuje dwa kluczowe aspekty przetwarzania. Pierwszy aspekt dotyczy czasu wykonania funkcji konwersji znajdujących się w bibliotekach DLL – jest to czas rzeczywistego przekształcania danych obrazu w reprezentację ASCII, obejmujący operacje mapowania wartości jasności pikseli na znaki z predefiniowanej palety. Drugi aspekt odnosi się do całkowitego czasu działania aplikacji, który oprócz czasu konwersji DLL uwzględnia również operacje wejścia-wyjścia, ładowanie bibliotek, przygotowanie danych oraz renderowanie wyników. Kluczowe jest zrozumienie, że wielowątkowość w kontekście tego projektu wpływa wyłącznie na całkowity czas działania programu poprzez równoległe przetwarzanie wielu obrazów. Sama funkcjonalność zawarta w bibliotekach DLL pozostaje jednowątkowa i jej czas wykonania nie ulega skróceniu przez zwiększenie liczby wątków – wielowątkowość jedynie pozwala na jednoczesne wywoływanie tej funkcji dla różnych obrazów. Pomiary przeprowadzono na trzech zestawach danych różnej wielkości: S (5 obrazów), M (20 obrazów) i L (50 obrazów), przy czym dla każdej konfiguracji wykonano pięć pomiarów w celu zapewnienia wiarygodności statystycznej. Testowano implementację asemblerową wykorzystującą instrukcje SSE4.1, implementację C++ bez optymalizacji oraz implementację C++ z flagą optymalizacji kompilatora O2.

6.1 Analiza czasów wykonania całego programu

6.1.1 Zestaw danych S (5 obrazów)

Tab. 1. Zestawienie czasów wykonania całego programu dla kolekcji danych S z DLL w wersji ASM

Wątki	**1**	**2**	**4**	**8**	**16**	**32**	**64**
Pomiar 1, ms	13429.1	8394.05	8533.5	6203.65	5442.74	6390.82	6561.22
Pomiar 2, ms	12672.4	8263.04	8459.54	5800.52	6657.27	5886.11	6506.67
Pomiar 3, ms	13180.9	8955.08	8491.85	6227.55	6451.88	5001.44	6018.85
Pomiar 4, ms	13542.3	8638.12	8315.17	5567.92	5617.31	6468.13	6240.11
Pomiar 5, ms	14834.3	8662.39	8527.56	6620.04	5770.33	6048.2	6270.42
Średnia, ms	**13531.8**	**8582.536**	**8465.524**	**6083.936**	**5987.906**	**5958.94**	**6319.454**

Tabela 1 przedstawia czasy wykonania całego programu dla kolekcji danych S z wykorzystaniem biblioteki DLL w wersji asemblerowej. Dla pojedynczego wątku średni czas wynosił 13531,8 ms. Zwiększenie liczby wątków do dwóch spowodowało znaczące przyspieszenie do 8582,536 ms, czyli o około 37% w stosunku do wariantu jednowątkowego. Dalsze zwiększanie liczby wątków do czterech, ośmiu i szesnastu przynosiło stopniową poprawę wydajności, przy czym najlepszy wynik uzyskano dla ośmiu wątków. Przy bardzo dużej liczbie wątków – dla 32 i 64 wątków czasy nieznacznie wzrosły w porównaniu do 16 wątków.

Tab. 2. Zestawienie czasów wykonania całego programu dla kolekcji danych S z DLL w wersji CPP

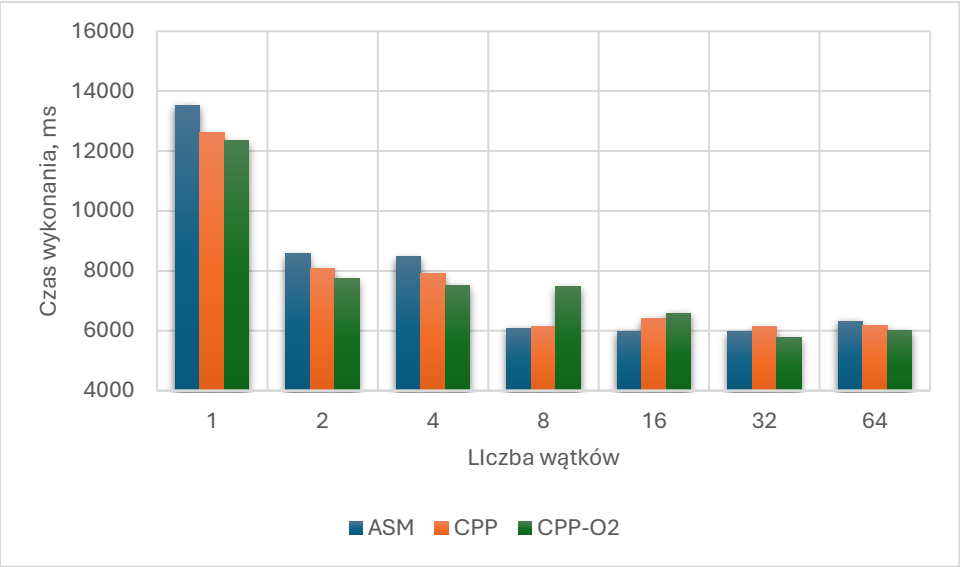
Wątki	**1**	**2**	**4**	**8**	**16**	**32**	**64**
Pomiar 1, ms	13018.8	7536.31	7626.61	6411.85	5314.59	5714.59	6362.76
Pomiar 2, ms	12452.4	8498.25	8217.58	5890.73	6658	6310.09	6416.8
Pomiar 3, ms	12195.8	8075.1	7332.39	5754.26	6211.85	6565.69	5879.7
Pomiar 4, ms	12993.9	7960.28	8315.13	6461.73	6074	6232.3	6031.43
Pomiar 5, ms	12380.4	8377.01	8010.54	6147.2	7781.44	5822.14	6130
Średnia, ms	**12608.26**	**8089.39**	**7900.45**	**6133.154**	**6407.976**	**6128.962**	**6164.138**

Tabela 2 zawiera analogiczne pomiary dla biblioteki DLL w wersji C++ bez optymalizacji kompilatora. Średni czas dla jednego wątku wyniósł 12608,26 ms, co jest nieznacznie lepszym wynikiem niż w wersji asemblerowej. Charakterystyka zależności od liczby wątków jest podobna do wersji asemblerowej – najlepsze wyniki osiągnięto dla 8-16 wątków, z minimalnym czasem 6133,154 ms dla ośmiu wątków. Warto zauważyć, że przy większej liczbie wątków (32-64) czasy stabilizują się na poziomie około 6100-6200 ms.

Tab. 3. Zestawienie czasów wykonania całego programu dla kolekcji danych S z DLL w wersji CPP z optymalizacją O2

Wątki	**1**	**2**	**4**	**8**	**16**	**32**	**64**
Pomiar 1, ms	12337.8	7719.31	7475.38	7484.61	6601.27	5769.09	5977.85
Pomiar 2, ms	12502.4	7851.67	7699.01	7521.83	6544.91	5902.18	6023.71
Pomiar 3, ms	12197.6	7608.45	7342.54	7396.02	6677.1	5689.73	5939.42
Pomiar 4, ms	12415.2	7793.88	7610.33	7532.74	6510.28	5803.65	6122.19
Pomiar 5, ms	12289.7	7681.22	7464.17	7441.35	6592.03	5728.44	6044.88
Średnia, ms	**12348.54**	**7730.906**	**7518.286**	**7475.31**	**6585.118**	**5778.618**	**6021.61**

Tabela 3 prezentuje wyniki dla biblioteki C++ skompilowanej z flagą optymalizacji O2, która znacząco poprawia wydajność kodu generowanego przez kompilator. Średni czas dla jednego wątku wyniósł 12348,54 ms, co jest najlepszym wynikiem spośród wszystkich trzech implementacji dla wariantu jednowątkowego.



Rys. 13. Wykres zależności średnich czasów wykonania całego programu dla kolekcji danych S od liczby wątków z podziałem na wersje DLL

Rysunek 13 wizualizuje porównanie średnich czasów wykonania całego programu dla wszystkich trzech wersji bibliotek w funkcji liczby wątków. Wykres wyraźnie pokazuje, że wszystkie implementacje wykazują podobny ogólny trend – gwałtowny spadek czasu przy przejściu z jednego do dwóch wątków, stopniową poprawę przy zwiększaniu liczby wątków do 8-16, a następnie stabilizację lub lekki wzrost dla bardzo dużej liczby wątków. Różnice między implementacjami stają się bardziej widoczne dopiero przy analizie czasów czystego wykonania funkcji DLL, co zostanie omówione w kolejnych rozdziałach raportu.

6.1.2 Zestaw danych M (20 obrazów)

Tab. 4. Zestawienie czasów wykonania całego programu dla kolekcji danych M z DLL w wersji ASM

Wątki	**1**	**2**	**4**	**8**	**16**	**32**	**64**
Pomiar 1, ms	53284.48	36062.07	24022.82	21683.93	21062.31	18566.84	20669.04
Pomiar 2, ms	52380.97	35353.42	26111.53	21114.40	21002.94	17815.83	19165.81
Pomiar 3, ms	53326.88	33298.72	26241.52	20862.55	19695.40	16615.55	19314.89
Pomiar 4, ms	50968.52	34800.20	23412.27	21868.40	21264.99	16530.75	20568.20
Pomiar 5, ms	54694.19	34008.54	25477.10	21359.04	21434.06	17114.82	19040.94
Średnia, ms	**52931.01**	**34704.59**	**25053.05**	**21377.66**	**20891.94**	**17328.76**	**19751.78**

Tabela 4 przedstawia czasy wykonania całego programu dla kolekcji danych M z wykorzystaniem biblioteki asemblerowej. Średni czas dla jednego wątku wyniósł 52931,01 ms, co stanowi prawie czterokrotny wzrost w porównaniu do zestawu S, odzwierciedlając proporcjonalnie większą ilość

danych do przetworzenia. Zwiększenie liczby wątków przyniosło tutaj znacznie bardziej wyraźne efekty niż w przypadku mniejszego zestawu danych – przejście z jednego do dwóch wątków skróciło czas do 34704,59 ms (redukcja o około 34%), a dalsze zwiększanie liczby wątków konsekwentnie poprawiało wydajność.

Tab. 5. Zestawienie czasów wykonania całego programu dla kolekcji danych M z DLL w wersji CPP

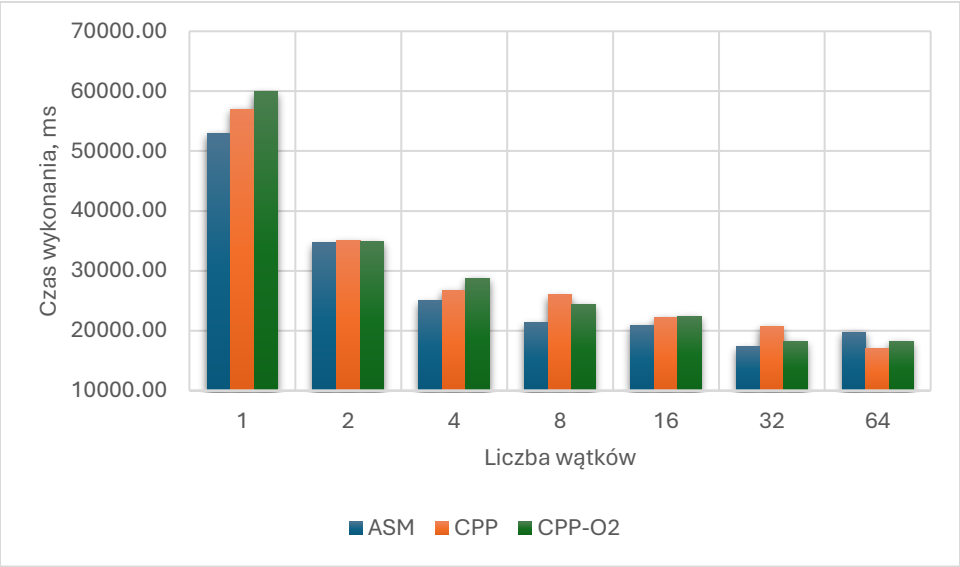
Wątki	**1**	**2**	**4**	**8**	**16**	**32**	**64**
Pomiar 1, ms	55135.553	34608.321	26933.631	26637.559	22369.967	21224.228	17277.966
Pomiar 2, ms	58039.718	34250.394	25911.601	24763.311	22360.409	19902.958	17001.105
Pomiar 3, ms	58031.705	34585.164	26171.608	25588.162	22412.997	21362.413	16866.594
Pomiar 4, ms	59157.632	36769.074	26612.493	25180.709	22201.761	20468.99	17001.608
Pomiar 5, ms	54413.574	35064.239	28042.959	27835.211	21884.263	20812.866	16926.793
Średnia, ms	**56955.636**	**35055.438**	**26734.458**	**26000.99**	**22245.879**	**20754.291**	**17014.813**

Tabela 5 zawiera pomiary dla implementacji C++ bez optymalizacji. Średni czas dla jednego wątku wyniósł 56955,636 ms, co jest o około 8% gorszym wynikiem niż w wersji asemblerowej.

Tab. 6. Zestawienie czasów wykonania całego programu dla kolekcji danych M z DLL w wersji CPP z optymalizacją O2

Wątki	**1**	**2**	**4**	**8**	**16**	**32**	**64**
Pomiar 1, ms	59924.1	34897.2	28733.4	24431.1	22472.1	18198.1	18194.2
Pomiar 2, ms	60381.6	35349.9	29210.7	24789.3	22951.4	18632.4	18332.7
Pomiar 3, ms	59112.5	34058.3	28029.1	24172.8	21887.6	17773.5	18018.3
Pomiar 4, ms	60742.3	35577.4	29356.8	24618.7	22311.2	18461.8	18544.1
Pomiar 5, ms	59602.8	34692.1	28640.5	24287.9	22504.9	18225.9	18263.6
Średnia, ms	**59952.66**	**34914.98**	**28794.1**	**24459.96**	**22425.44**	**18258.34**	**18270.58**

Tabela 6 prezentuje wyniki dla biblioteki C++ z optymalizacją O2. Średni czas dla jednego wątku wyniósł 59952,66 ms, co jest najgorszym wynikiem spośród wszystkich trzech implementacji dla zestawu M. Pomimo wolniejszego startu, implementacja z optymalizacją O2 wykazuje bardzo dobrą skalowalność – przyspieszenie przy przejściu z 1 do 64 wątków jest około 3,3-krotne.



Rys. 14. Wykres zależności średnich czasów wykonania całego programu dla kolekcji danych M od liczby wątków z podziałem na wersje DLL

Rysunek 14 ilustruje porównanie wszystkich trzech implementacji dla zestawu danych M. Wykres wyraźnie pokazuje, że dla pojedynczego wątku implementacja asemblerowa oferuje najlepszą wydajność. Wraz ze wzrostem liczby wątków różnice te stopniowo zanikają, a wszystkie słupki zbiegają się do podobnych wartości dla 32-64 wątków.

6.1.3 Zestaw danych L (50 obrazów)

Tab. 7. Zestawienie czasów wykonania całego programu dla kolekcji danych L z DLL w wersji ASM

Wątki	**1**	**2**	**4**	**8**	**16**	**32**	**64**
Pomiar 1, ms	92483.40	70293.10	52468.20	39651.50	37959.20	35683.50	28986.40
Pomiar 2, ms	92301.70	69411.60	53820.40	40821.20	38214.00	36101.80	29287.90
Pomiar 3, ms	92677.90	71129.40	51392.00	38990.10	37774.30	35107.20	29105.50
Pomiar 4, ms	91822.30	69783.50	52206.70	39412.80	38122.90	35355.40	28794.10
Pomiar 5, ms	93531.60	70457.90	52514.10	40173.90	37562.70	36210.40	29368.60
Średnia, ms	**92563.38**	**70215.10**	**52480.28**	**39809.90**	**37926.62**	**35691.66**	**29108.50**

Tabela 7 przedstawia czasy wykonania dla największego zestawu danych L z wykorzystaniem biblioteki asemblerowej. Średni czas dla jednego wątku wyniósł 92563,38 ms, co stanowi niemal dwukrotny wzrost względem zestawu M i siedmiokrotny względem zestawu S. Przejście z jednego do dwóch wątków skróciło czas do 70215,10 ms (redukcja o 24%). Charakterystyczne jest bardzo wyraźne obniżenie czasów przy każdym zwiększeniu liczby wątków. Świadczy to o tym, że duży zestaw

danych L zawiera wystarczającą ilość pracy do przetworzenia, aby efektywnie wykorzystać wszystkie dostępne wątki bez znaczących narzutów związanych z zarządzaniem nimi.

Tab. 8. Zestawienie czasów wykonania całego programu dla kolekcji danych L z DLL w wersji CPP

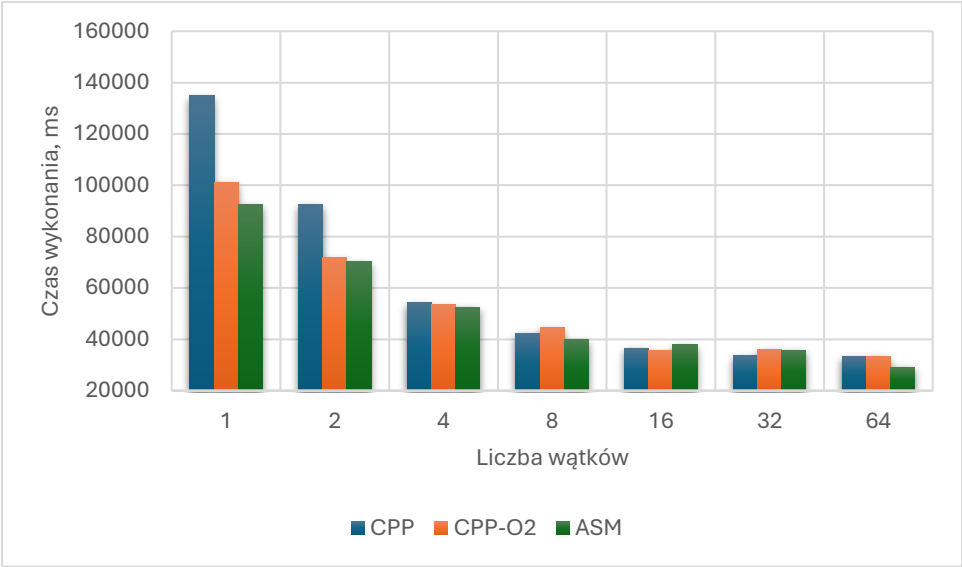
Wątki	**1**	**2**	**4**	**8**	**16**	**32**	**64**
Pomiar 1, ms	134952	92564.8	54210.5	42434.5	36443.6	33722.3	33296.7
Pomiar 2, ms	135780	93112.2	55362.8	41698.7	37201.9	34110.8	32984.1
Pomiar 3, ms	133894	91831.6	53107.4	43102.5	35987.4	33397.6	33529.2
Pomiar 4, ms	135120	92988.5	54790.3	42783.6	36680.5	33822.4	32876.8
Pomiar 5, ms	134501	92130.4	53844.2	42066.1	36219.7	33540.9	33483.5
Średnia, ms	**134849.4**	**92525.5**	**54263.04**	**42417.08**	**36506.62**	**33718.8**	**33234.06**

Tabela 8 zawiera pomiary dla implementacji C++ bez optymalizacji. Średni czas dla jednego wątku wyniósł 134849,4 ms, co jest znacząco gorszym wynikiem niż w wersji asemblerowej – różnica wynosi aż 42286 ms, czyli około 46%. Jest to pierwsza sytuacja, w której implementacja C++ bez optymalizacji wyraźnie odstaje od wersji asemblerowej w kontekście całkowitego czasu działania programu.

Tab. 9. Zestawienie czasów wykonania całego programu dla kolekcji danych L z DLL w wersji CPP z optymalizacją O2

Wątki	**1**	**2**	**4**	**8**	**16**	**32**	**64**
Pomiar 1, ms	100814	71637.8	53476.8	44149	35505.8	35938.8	33454.2
Pomiar 2, ms	101902.5	72988.3	54621.4	44892.7	36272.1	36514.6	33810.5
Pomiar 3, ms	99872.1	70542.6	52333.9	43721.5	34918.4	35292.7	33027.4
Pomiar 4, ms	102511.3	72391.4	54055.2	45088.1	35809.7	36173.2	33688.9
Pomiar 5, ms	99988.6	71228.9	53024.7	44415.3	35266.2	35701.3	33210.8
Średnia, ms	**101017.7**	**71757.8**	**53502.4**	**44453.32**	**35554.44**	**35924.12**	**33438.36**

Tabela 9 prezentuje wyniki dla biblioteki C++ z optymalizacją O2. Średni czas dla jednego wątku wyniósł 101017,7 ms, co jest wartością pośrednią między implementacją asemblerową a C++ bez optymalizacji. Optymalizacja kompilatora wyraźnie poprawiła wydajność względem wariantu nieoptymalizowanego (o około 25%).



Rys. 15. Wykres zależności średnich czasów wykonania całego programu dla kolekcji danych L od liczby wątków z podziałem na wersje DLL

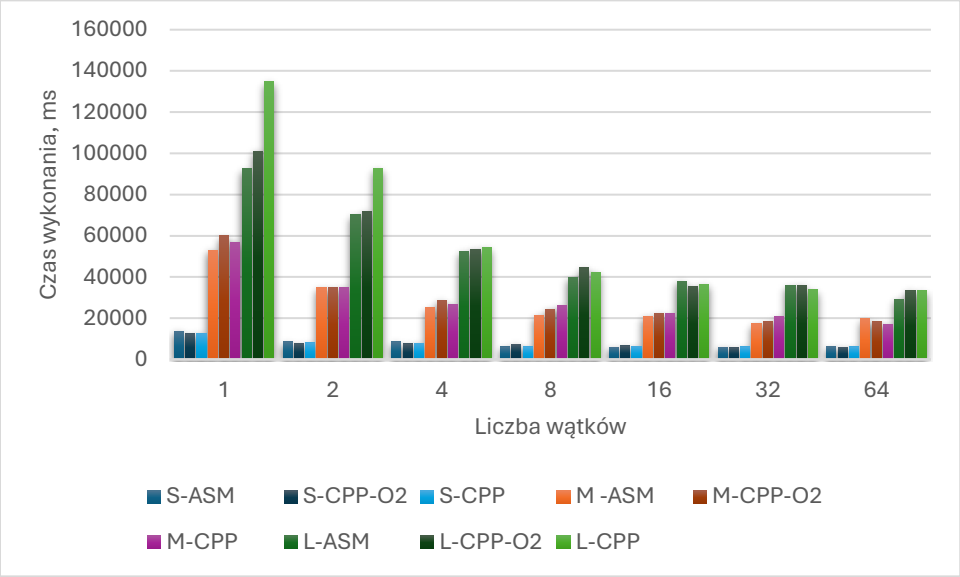
Rysunek 15 wizualizuje porównanie wszystkich trzech implementacji dla zestawu danych L. Wykres wyraźnie pokazuje, że implementacja asemblerowa oferuje najlepsze czasy dla większości konfiguracji wątkowych, implementacja C++ z optymalizacją O2 zajmuje pozycję środkową, a implementacja C++ bez optymalizacji jest najwolniejsza. Wszystkie trzy implementacje wykazują doskonałą skalowalność. Szczególnie zestaw L, który charakteryzuje się najwyraźniejszym spadkiem wysokości słupków.

6.1.4 Porównanie zbiorowe wszystkich zestawów danych

Tab. 10. Zestawienie średnich czasów wykonania całego programu dla kolekcji danych S, M, L z podziałem na wersje DLL

Wątki	**1**	**2**	**4**	**8**	**16**	**32**	**64**
S-ASM	13531.8	8582.536	8465.524	6083.936	5987.906	5958.94	6319.454
S-CPP-O2	12348.54	7730.906	7518.286	7475.31	6585.118	5778.618	6021.61
S-CPP	12608.26	8089.39	7900.45	6133.154	6407.976	6128.962	6164.138
M -ASM	52931.01	34704.59	25053.05	21377.66	20891.94	17328.76	19751.78
M-CPP-O2	59952.66	34914.98	28794.1	24459.96	22425.44	18258.34	18270.58
M-CPP	56955.64	35055.44	26734.46	26000.99	22245.88	20754.29	17014.81
L-ASM	92563.38	70215.1	52480.28	39809.9	37926.62	35691.66	29108.5
L-CPP-O2	101017.7	71757.8	53502.4	44453.32	35554.44	35924.12	33438.36
L-CPP	134849.4	92525.5	54263.04	42417.08	36506.62	33718.8	33234.06

Tabela 10 przedstawia kompleksowe zestawienie średnich czasów wykonania całego programu dla wszystkich trzech zestawów danych i wszystkich trzech implementacji bibliotek. Dla zestawu S różnice między implementacjami są minimalne i mieszczą się w przedziale 12348,54-13531,8 ms dla jednego wątku, co stanowi mniej niż 10% różnicy. Dla zestawu M różnice te wzrastają do 52931,01-59952,66 ms (około 13% różnicy), a dla zestawu L osiągają maksymalny poziom 92563,38-134849,4 ms (około 46% różnicy). Potwierdza to, że wraz ze wzrostem rozmiaru przetwarzanych danych znaczenie optymalizacji niskopoziomowych rośnie, a różnice między implementacjami stają się bardziej wyraźne. Dla wszystkich zestawów danych optymalna liczba wątków wynosi 8-16, co koreluje z typową architekturą procesorów wielordzeniowych.



Rys. 16. Wykres zależności średnich czasów wykonania całego programu dla kolekcji danych S, M, L od liczby wątków z podziałem na wersje DLL

Rysunek 16 ilustruje graficzne porównanie wszystkich dziewięciu słupków wydajności jednocześnie. Wyraźnie widoczne jest grupowanie słupków według rozmiaru zestawu danych – wszystkie trzy implementacje dla zestawu S tworzą dolną grupę słupków, dla zestawu M środkową, a dla zestawu L górną. Wewnątrz każdej grupy słupki dla różnych implementacji są do siebie bardzo zbliżone dla małej liczby wątków i stopniowo się zbiegają przy zwiększaniu liczby wątków. Spadek wysokości słupków rośnie wraz z rozmiarem zestawu danych, co ilustruje lepszą skalowalność wielowątkowości dla większych ilości danych do przetworzenia. Wykres jednoznacznie potwierdza, że dla całkowitego czasu działania programu rozmiar danych jest czynnikiem dominującym nad wyborem implementacji algorytmu – różnice wynikające z rozmiaru zestawu są wielokrotnie większe niż różnice między implementacjami asemblerową i C++.

6.2 Analiza czasów wykonania bibliotek DLL

6.2.1 Zestaw danych S (5 obrazów)

Tab. 11. Zestawienie czasów wykonania DLL w wersji ASM dla kolekcji danych S

Wątki	1	2	4	8	16	32	64
Pomiar 1, ms	0.3463	0.9419	1.3163	1.5421	1.7652	1.4421	4.7631
Pomiar 2, ms	0.5775	1.1787	1.4394	1.0074	1.5291	1.414	2.5518
Pomiar 3, ms	0.532	0.7521	1.5428	1.0756	2.5197	1.2994	3.5767
Pomiar 4, ms	0.8606	0.7833	1.0623	0.9624	1.0946	1.7066	2.8615
Pomiar 5, ms	0.7105	0.7911	0.7913	1.0797	1.1466	1.3233	3.493
Średnia, ms	**0.60538**	**0.88942**	**1.23042**	**1.13344**	**1.61104**	**1.43708**	**3.44922**

Tabela 11 przedstawia czasy czystego wykonania funkcji konwersji w bibliotece DLL w wersji asemblerowej dla zestawu danych S. Wyniki te dotyczą wyłącznie czasu spędzonego wewnątrz funkcji konwersji, bez uwzględnienia operacji wejścia-wyjścia, ładowania obrazów czy renderowania wyników. Średni czas dla jednego wątku wyniósł zaledwie 0,60538 ms, co stanowi mniej niż 0,005% całkowitego czasu działania programu dla tego zestawu. To ekstremalna dysproporcja jednoznacznie pokazuje, że w przypadku małych obrazów czas samej konwersji jest znikomy w porównaniu do operacji pomocniczych. Co kluczowe, zwiększanie liczby wątków nie powoduje skrócenia tego czasu – wręcz przeciwnie, obserwujemy stopniowy wzrost do 1,61104 ms dla 16 wątków i dalszy wzrost do 3,44922 ms dla 64 wątków. Zjawisko to jest bezpośrednim dowodem na to, że wielowątkowość nie przyspiesza wykonania samej funkcji DLL, a obserwowany wzrost czasów wynika z narzutów związanych z zarządzaniem wieloma jednoczesnymi wywołaniami funkcji, konkurencją o pamięć cache procesora oraz koniecznością synchronizacji dostępu do współdzielonych zasobów.

Tab. 12. Zestawienie czasów wykonania DLL w wersji CPP dla kolekcji danych S

Wątki	1	2	4	8	16	32	64
Pomiar 1, ms	8.5484	7.3078	15.3435	22.4118	24.4841	32.9366	49.5892
Pomiar 2, ms	9.1458	7.8873	15.4145	32.2313	22.3881	41.166	48.9168
Pomiar 3, ms	7.8237	8.0684	16.7239	36.9079	36.7785	27.2576	46.2038
Pomiar 4, ms	9.4669	8.4541	20.1103	22.8745	30.422	35.848	39.3288
Pomiar 5, ms	8.4485	8.9318	21.1799	38.0637	36.0071	15.0284	50.8376
Średnia, ms	**8.68666**	**8.12988**	**17.75442**	**30.49784**	**30.01596**	**30.44732**	**46.97524**

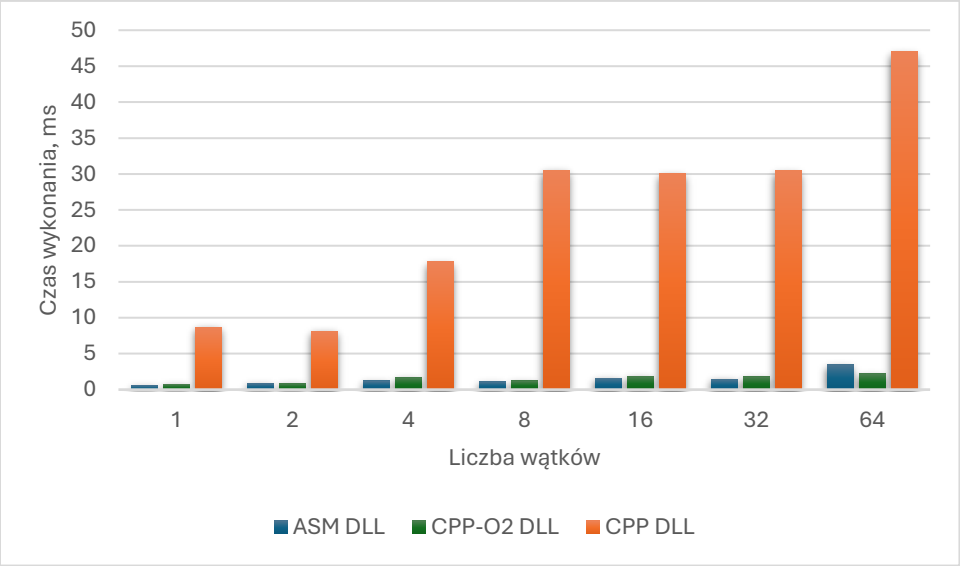
Tabela 12 zawiera analogiczne pomiary dla implementacji C++ bez optymalizacji. Średni czas dla jednego wątku wyniósł 8,68666 ms, co jest ponad 14-krotnie gorszym wynikiem niż w wersji asemblerowej. Ta znacząca różnica ilustruje przewagę implementacji asemblerowej wykorzystującej instrukcje SIMD SSE4.1 nad kodem C++ generowanym przez kompilator bez optymalizacji. Podobnie jak w wersji asemblerowej, obserwujemy wzrost czasów wraz ze zwiększaniem liczby wątków – od

8,68666 ms dla jednego wątka do 46,97524 ms dla 64 wątków, czyli ponad pięciokrotny wzrost. Potwierdza to, że narzuty związane z wielowątkowością są uniwersalnym zjawiskiem niezależnym od implementacji algorytmu.

Tab. 13. Zestawienie czasów wykonania DLL w wersji CPP z optymalizacją O2 dla kolekcji danych S

Wątki	**1**	**2**	**4**	**8**	**16**	**32**	**64**
Pomiar 1, ms	0.7713	0.8692	1.6712	1.2197	1.8492	1.7834	2.2204
Pomiar 2, ms	0.7584	0.8826	1.7054	1.2481	1.8731	1.8119	2.1978
Pomiar 3, ms	0.7655	0.8547	1.6428	1.2934	1.8013	1.7655	2.1589
Pomiar 4, ms	0.7826	0.8771	1.689	1.2321	1.8295	1.7998	2.2416
Pomiar 5, ms	0.7691	0.8614	1.6583	1.2105	1.8467	1.7842	2.2037
Średnia, ms	**0.76938**	**0.869**	**1.67334**	**1.24076**	**1.83996**	**1.78896**	**2.20448**

Tabela 13 prezentuje wyniki dla biblioteki C++ z optymalizacją O2. Średni czas dla jednego wątku wyniósł 0,76938 ms, co jest wynikiem niemal identycznym jak dla implementacji asemblerowej (0,60538 ms) i ponad 11-krotnie lepszym niż dla C++ bez optymalizacji. To przyspieszenie ilustruje możliwości nowoczesnych kompilatorów, które potrafią generować kod maszynowy o wydajności zbliżonej do ręcznie pisanego assemblera, pod warunkiem zastosowania odpowiednich flag optymalizacji.



Rys. 17. Wykres zależności średnich czasów wykonania DLL w różnych wersjach dla kolekcji danych S od liczby wątków

Rysunek 17 wizualizuje porównanie czasów wykonania DLL dla wszystkich trzech implementacji. Wykres ilustruje przepaść wydajnościową między implementacją C++ bez optymalizacji a

pozostałymi dwoma wersjami - słupki dla C++ bez optymalizacji są wyraźnie odseparowane i osiągają wartości kilkukrotnie wyższe. Wysokości słupków dla implementacji asemblerowej i C++ z optymalizacją O2 są do siebie bardzo zbliżone. Wszystkie słupki wykazują tendencję wzrostową, co jednoznacznie potwierdza, że wielowątkowość zwiększa czasy wykonania samych funkcji DLL poprzez wprowadzanie narzutów synchronizacyjnych i konkurencji o zasoby. Wykres ten stanowi kluczowy element rozumienia architektury systemu – pokazuje, że przyspieszenie całkowitego czasu działania programu przez wielowątkowość nie wynika z przyspieszenia samych obliczeń konwersji, lecz z równoległego przetwarzania wielu niezależnych obrazów.

6.2.2 Zestaw danych M (20 obrazów)

Tab. 14. Zestawienie czasów wykonania DLL w wersji ASM dla kolekcji danych M

Wątki	**1**	**2**	**4**	**8**	**16**	**32**	**64**
Pomiar 1, ms	1.753	2.357	4.492	4.091	4.5	5.501	5.748
Pomiar 2, ms	1.791	2.171	4.697	4.938	4.362	5.256	5.886
Pomiar 3, ms	1.92	2.322	4.489	4.922	4.858	5.759	5.725
Pomiar 4, ms	1.758	2.377	5.192	4.642	4.405	5.732	5.912
Pomiar 5, ms	1.744	2.434	5.177	4.472	4.83	5.31	5.613
Średnia, ms	**1.7932**	**2.3322**	**4.8094**	**4.613**	**4.591**	**5.5116**	**5.7768**

Tabela 14 przedstawia czasy wykonania DLL w wersji asemblerowej dla zestawu danych M. Średni czas dla jednego wątku wyniósł 1,7932 ms, co stanowi prawie trzykrotny wzrost względem zestawu S, odzwierciedlając proporcjonalnie większą ilość danych do przetworzenia. Podobnie jak dla zestawu S, obserwujemy wzrost czasów wraz ze zwiększaniem liczby wątków.

Tab. 15. Zestawienie czasów wykonania DLL w wersji CPP dla kolekcji danych M

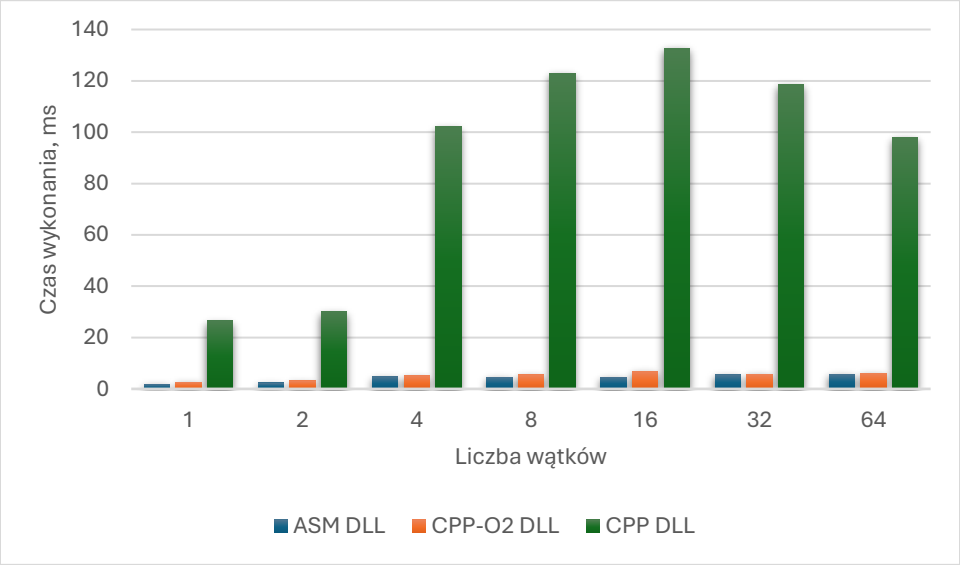
Wątki	**1**	**2**	**4**	**8**	**16**	**32**	**64**
Pomiar 1, ms	25.188	29.509	103.366	128.024	133.832	123.077	100.575
Pomiar 2, ms	27.444	28.989	96.845	113.138	133.739	110.591	97.902
Pomiar 3, ms	27.438	29.476	98.504	119.689	134.251	124.383	96.604
Pomiar 4, ms	28.313	32.65	101.317	116.453	132.194	115.94	97.907
Pomiar 5, ms	24.627	30.172	110.445	137.536	129.102	119.19	97.185
Średnia, ms	**26.602**	**30.1592**	**102.0954**	**122.968**	**132.6236**	**118.6362**	**98.0346**

Tabela 15 zawiera pomiary dla implementacji C++ bez optymalizacji. Średni czas dla jednego wątku wyniósł 26,602 ms, co jest prawie 15-krotnie gorszym wynikiem niż w wersji asemblerowej. Różnica ta jest bardzo podobna do obserwowanej dla zestawu S, co potwierdza, że względna przewaga implementacji asemblerowej jest niezależna od rozmiaru przetwarzanych danych i wynika z fundamentalnych różnic w generowanym kodzie maszynowym.

Tab. 16. Zestawienie czasów wykonania DLL w wersji CPP z optymalizacją O2 dla kolekcji danych M

Wątki	1	2	4	8	16	32	64
Pomiar 1, ms	2.6635	3.1491	5.1195	5.4716	6.7898	5.6473	6.1466
Pomiar 2, ms	2.7042	3.2138	5.2879	5.6021	6.9124	5.6889	6.2174
Pomiar 3, ms	2.6511	3.0876	5.0612	5.384	6.7015	5.8988	6.0897
Pomiar 4, ms	2.7327	3.2459	5.3415	5.5333	6.8542	5.9295	6.2568
Pomiar 5, ms	2.6843	3.1714	5.2057	5.4598	6.781	5.7746	6.1981
Średnia, ms	**2.68716**	**3.17356**	**5.20316**	**5.49016**	**6.80778**	**5.78782**	**6.18172**

Tabela 16 prezentuje wyniki dla biblioteki C++ z optymalizacją O2. Średni czas dla jednego wątku wyniósł 2,68716 ms, co jest około 50% gorszym wynikiem niż w wersji asemblerowej, ale niemal 10-krotnie lepszym niż w wersji C++ bez optymalizacji. Względna przewaga wersji asemblerowej nad optymalizowanym C++ jest większa dla zestawu M niż dla zestawu S, co sugeruje, że dla większych ilości danych optymalizacje niskopoziomowe wykonane ręcznie w assemblerze dają większe korzyści niż automatyczne optymalizacje kompilatora.



Rys. 18. Wykres zależności średnich czasów wykonania DLL w różnych wersjach dla kolekcji danych M od liczby wątków

Rysunek 18 ilustruje porównanie wszystkich trzech implementacji dla zestawu M. Wykres wyraźnie pokazuje różnicę między implementacją C++ bez optymalizacji a pozostałymi dwoma wersjami – słupki dla C++ bez optymalizacji osiąga wartości rzędu 130 ms, podczas gdy pozostałe słupki mieszczą się w zakresie 2-7 ms. Słupki dla implementacji asemblerowej i C++ z optymalizacją O2 osiągają porównywalne wysokości, przy czym wersja asemblerowa konsekwentnie oferuje lepszą wydajność. Wszystkie słupki wykazują tendencję wzrostową, potwierdzając uniwersalny charakter narzutów związanych z wielowątkowością.

6.2.3 Zestaw danych M (50 obrazów)

Tab. 17. Zestawienie czasów wykonania DLL w wersji ASM dla kolekcji danych L

Wątki	1	2	4	8	16	32	64
Pomiar 1, ms	4.4049	5.3466	7.2607	11.0965	12.3949	11.0911	11.2472
Pomiar 2, ms	4.5121	5.1924	7.3112	11.5427	12.0083	11.3058	11.4103
Pomiar 3, ms	4.3865	5.3792	7.2019	10.9046	12.6674	10.9887	11.3256
Pomiar 4, ms	4.5688	5.4237	7.4382	11.2874	12.4235	11.1195	10.9977
Pomiar 5, ms	4.4912	5.2911	7.335	11.0168	12.519	11.2643	11.3861
Średnia, ms	**4.4727**	**5.3266**	**7.3094**	**11.1696**	**12.40262**	**11.15388**	**11.27338**

Tabela 17 przedstawia czasy wykonania DLL w wersji asemblerowej dla największego zestawu danych L. Średni czas dla jednego wątku wyniósł 4,4727 ms, co stanowi około 2,5-krotny wzrost względem zestawu M, odzwierciedlając dalszy wzrost ilości danych do przetworzenia. Podobnie jak dla mniejszych zestawów, obserwujemy wzrost czasów wraz ze zwiększaniem liczby wątków – dla 64 wątków uzyskano 11,27338 ms, co stanowi około 2,5-krotny wzrost.

Tab. 18. Zestawienie czasów wykonania DLL w wersji CPP dla kolekcji danych L

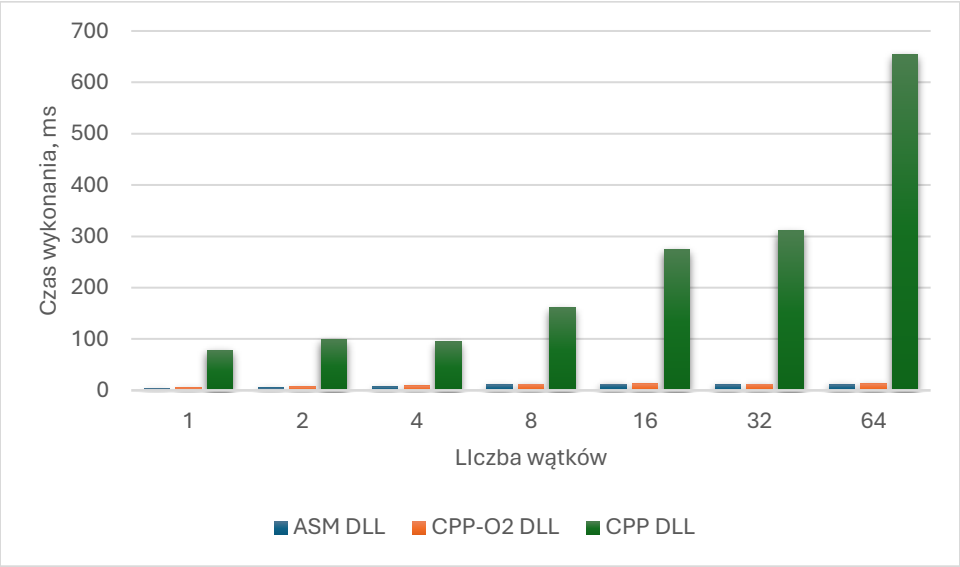
Wątki	1	2	4	8	16	32	64
Pomiar 1, ms	78.3998	98.245	94.3959	160.331	275.336	311.556	654.598
Pomiar 2, ms	79.2103	97.112	91.8427	158.245	273.908	304.774	645.928
Pomiar 3, ms	77.5041	99.584	96.1224	163.017	270.414	315.021	662.744
Pomiar 4, ms	78.9635	98.766	95.5778	159.442	279.103	308.517	649.33
Pomiar 5, ms	76.8852	96.902	93.7189	161.891	272.019	313.008	658.112
Średnia, ms	**78.19258**	**98.1218**	**94.33154**	**160.5852**	**274.156**	**310.5752**	**654.1424**

Tabela 18 zawiera pomiary dla implementacji C++ bez optymalizacji. Średni czas dla jednego wątku wyniósł 78,19258 ms, co jest niemal 17,5-krotnie gorszym wynikiem niż w wersji asemblerowej. Warto również zauważyć, że implementacja C++ bez optymalizacji osiąga 654,1424 ms dla 64 wątków, co stanowi ponad ośmiokrotny wzrost względem wariantu jednowątkowego.

Tab. 19. Zestawienie czasów wykonania DLL w wersji CPP z optymalizacją O2 dla kolekcji danych L

Wątki	1	2	4	8	16	32	64
Pomiar 1, ms	5.448	7.2402	8.532	11.9313	12.4012	12.065	12.5937
Pomiar 2, ms	5.517	7.3184	8.643	12.1125	12.5824	12.2034	12.7321
Pomiar 3, ms	5.399	7.1851	8.462	11.7748	12.2578	11.9812	12.4846
Pomiar 4, ms	5.561	7.3615	8.702	12.0439	12.5036	12.1768	12.6559
Pomiar 5, ms	5.472	7.2689	8.583	11.8964	12.3511	12.0123	12.5742
Średnia, ms	**5.4794**	**7.27482**	**8.5844**	**11.95178**	**12.41922**	**12.08774**	**12.6081**

Tabela 19 prezentuje wyniki dla biblioteki C++ z optymalizacją O2. Średni czas dla jednego wątku wyniósł 5,4794 ms, co jest około 23% gorszym wynikiem niż w wersji asemblerowej, ale ponad 14-krotnie lepszym niż w wersji C++ bez optymalizacji. Podobnie jak dla mniejszych zestawów, optymalizacja kompilatora przynosi przyspieszenie, choć nadal nie dorównuje ręcznie zoptymalizowanemu assemblerowi. Wzrost czasów wraz ze zwiększaniem liczby wątków jest bardzo umiarkowany – dla 64 wątków uzyskano 12,6081 ms, co stanowi zaledwie 2,3-krotny wzrost względem wariantu jednowątkowego.



Rys. 19. Wykres zależności średnich czasów wykonania DLL w różnych wersjach dla kolekcji danych L od liczby wątków

Rysunek 19 wizualizuje porównanie wszystkich trzech implementacji dla zestawu L. Wykres ilustruje przepaść wydajnościową między implementacją C++ bez optymalizacji a pozostałymi dwoma wersjami - słupki dla C++ bez optymalizacji szybko odbiegają od pozostałych i osiągają wartość ponad 650 ms dla 64 wątków, podczas gdy pozostałe słupki dla implementacji asemblerowej i C++ z optymalizacją O2 pozostają w przedziale 4-13 ms. Te dwie ostatnie serie słupków osiągają niemal

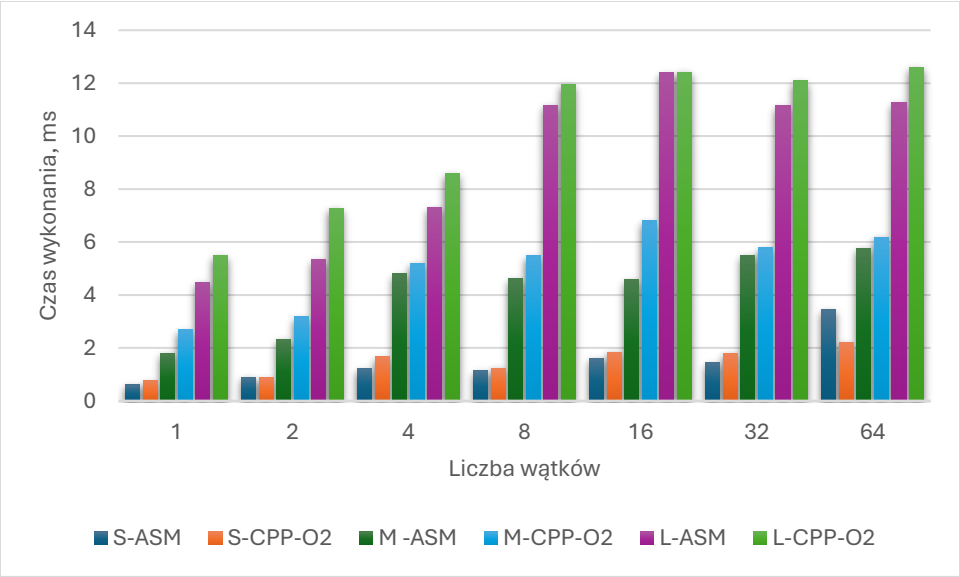
identyczne wysokości, z wersją asemblerową konsekwentnie oferującą około 20-25% lepszą wydajność.

6.2.4 Porównanie zbiorowe wszystkich zestawów danych

Tab. 20. Zestawienie średnich czasów wykonania DLL w wersji ASM oraz CPP z optymalizacją O2 dla kolekcji danych S,M,L

Wątki	1	2	4	8	16	32	64
S-ASM	0.60538	0.88942	1.23042	1.13344	1.61104	1.43708	3.44922
S-CPP-O2	0.76938	0.869	1.67334	1.24076	1.83996	1.78896	2.20448
M -ASM	1.7932	2.3322	4.8094	4.613	4.591	5.5116	5.7768
M-CPP-O2	2.68716	3.17356	5.20316	5.49016	6.80778	5.78782	6.18172
L-ASM	4.4727	5.3266	7.3094	11.1696	12.40262	11.15388	11.27338
L-CPP-O2	5.4794	7.27482	8.5844	11.95178	12.41922	12.08774	12.6081

Tabela 20 przedstawia kompleksowe zestawienie średnich czasów wykonania DLL dla implementacji asemblerowej oraz C++ z optymalizacją O2 dla wszystkich trzech zestawów danych. Zestawienie to pozwala na bezpośrednie porównanie najwydajniejszych implementacji i ocenę ich skalowania wraz z rozmiarem danych. Dla zestawu S różnica między implementacjami wynosi około 27% (0,60538 ms vs 0,76938 ms), dla zestawu M rośnie do około 50% (1,7932 ms vs 2,68716 ms), a dla zestawu L stabilizuje się na poziomie około 23% (4,4727 ms vs 5,4794 ms).



Rys. 20. Wykres zależności średnich czasów wykonania DLL w wersji ASM oraz CPP z optymalizacją O2 dla kolekcji danych S, M, L od liczby wątków

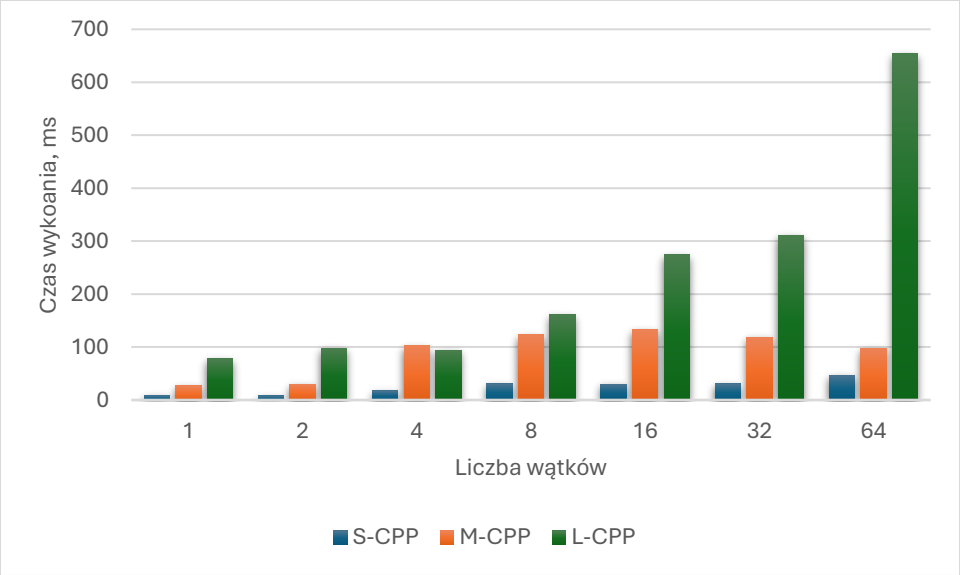
Rysunek 20 wizualizuje graficzne porównanie czasów wykonania DLL dla implementacji asemblerowej i C++ z optymalizacją O2 dla wszystkich zestawów danych. Wykres wyraźnie pokazuje

separację serii słupków według rozmiaru zestawu – wszystkie słupki dla zestawu S przebiegają w dolnej części wykresu (poniżej 4 ms), dla zestawu M w środkowej części (2-7 ms), a dla zestawu L w górnej części (4-13 ms). Wewnątrz każdej pary słupków dla danego zestawu, implementacja asemblerowa konsekwentnie oferuje lepszą wydajność. Wszystkie słupki wykazują tendencję wzrostową wraz ze zwiększaniem liczby wątków. Wykres jednoznacznie potwierdza, że wielowątkowość nie przyspiesza wykonania samych funkcji DLL – słupki idą w górę, a nie w dół, wraz ze wzrostem liczby wątków.

Tab. 21. Zestawienie średnich czasów wykonania DLL w wersji CPP dla kolekcji danych S, M, L

Wątki	**1**	**2**	**4**	**8**	**16**	**32**	**64**
S-CPP	8.68666	8.12988	17.75442	30.49784	30.01596	30.44732	46.97524
M-CPP	26.602	30.1592	102.0954	122.968	132.6236	118.6362	98.0346
L-CPP	78.19258	98.1218	94.33154	160.5852	274.156	310.5752	654.1424

Tabela 21 prezentuje czasy wykonania DLL wyłącznie dla implementacji C++ bez optymalizacji. To zestawienie służy jako punkt odniesienia ilustrujący, jakie mogą być konsekwencje braku optymalizacji w kodzie wykonującym intensywne obliczenia. Dla zestawu S czasy wahają się od 8,68666 ms do 46,97524 ms, dla zestawu M od 26,602 ms do 98,0346 ms, a dla zestawu L od 78,19258 ms do 654,1424 ms.



Rys. 21. Wykres zależności średnich czasów wykonania DLL w wersji CPP dla kolekcji danych S, M, L od liczby wątków

Rysunek 21 ilustruje graficzne porównanie czasów wykonania DLL dla implementacji C++ bez optymalizacji. Wyraźnie widoczne jest pogorszenie czasów wraz ze wzrostem zarówno rozmiaru danych, jak i liczby wątków - słupek dla zestawu L przy 64 wątkach osiąga wartość ponad 650 ms, podczas gdy dla zestawu S przy 1 wątku zaledwie około 9 ms.

7 Uruchamianie, testowanie i debugowanie

7.1 Uruchamianie aplikacji

Podstawowe uruchomienie aplikacji odbywa się poprzez wykonanie pliku wykonywalnego programu głównego znajdującego się w katalogu projektu. Aplikacja nie wymaga żadnych argumentów wiersza poleceń ani specjalnych uprawnień administracyjnych. Podczas startu aplikacja automatycznie inicjalizuje wszystkie wymagane komponenty systemowe, w tym bibliotekę OpenCV oraz wykrywa liczbę rdzeni logicznych procesora. Wykryta liczba rdzeni jest następnie ustawiana jako domyślna wartość suwaka kontrolującego liczbę wątków roboczych.

W momencie startu aplikacja automatycznie próbuje załadować domyślną bibliotekę DLL w wersji C++. System weryfikuje dostępność wymaganych plików bibliotek – *konwersja_cpp.dll* i *konwersja_asm.dll* – w katalogu roboczym aplikacji oraz sprawdza obecność wymaganych funkcji eksportowanych przez bibliotekę. Po pomyślnej inicjalizacji wszystkich komponentów prezentowany jest główny interfejs graficzny.

Przed rozpoczęciem przetwarzania użytkownik może opcjonalnie skonfigurować parametry działania aplikacji poprzez dostępne elementy sterujące. Przełączenie między trybami bibliotek DLL odbywa się poprzez kliknięcie odpowiedniego checkboxa "Tryb ASM" lub "Tryb CPP". Regulacja liczby wątków roboczych odbywa się poprzez przesuwanie suwaka "Watki" w zakresie od 1 do 64 wątków. Po podstawowej konfiguracji użytkownik ma do wyboru dwa główne tryby pracy aplikacji – Tryb pracy z pojedynczym obrazem lub Tryb pracy z całym folderem obrazów.

7.2 Testowanie i walidacji

7.2.1 Testowanie pojedynczych obrazów

Zestaw testowy obejmował obrazy o różnych rozdzielczościach, od małych 800×600 pikseli przez średnie 1920×1080 pikseli aż do dużych 3840×2160 pikseli. Dla każdego obrazu testowego wykonywano konwersję przy użyciu wszystkich trzech implementacji bibliotek DLL – asemblerowej, C++ bez optymalizacji oraz C++ z flagą O2 – co umożliwiło bezpośrednie porównanie wydajności różnych podejść implementacyjnych dla identycznych danych wejściowych. Każda konfiguracja była testowana z różnymi liczbami wątków roboczych, począwszy od pojedynczego wątku, poprzez typowe konfiguracje 2, 4, 8 i 16 wątków, aż do ekstremalnych przypadków 32 i 64 wątków testujących granice skalowalności systemu. Weryfikacja poprawności wyników obejmowała porównanie wizualne renderowanych obrazów ASCII dla różnych liczb wątków.

7.2.2 Testowanie folderów obrazów

Przygotowano trzy zestawy testowe o różnych rozmiarach – zestaw S zawierający 5 obrazów, zestaw M zawierający 20 obrazów oraz zestaw L zawierający 50 obrazów. Obrazy w każdym zestawie były celowo różnorodne pod względem zawartości, obejmując fotografie krajobrazowe z dominującymi jednolitymi obszarami, portrety z dużą szczegółowością oraz zdjęcia z wysokim kontrastem, co pozwoliło przetestować algorytm w różnych scenariuszach wizualnych. Struktura katalogów testowych była również zróżnicowana – niektóre testy wykorzystywały płaską strukturę z wszystkimi obrazami w jednym folderze, podczas gdy inne testowały rekurencyjne przeszukiwanie zagnieżdżonych podfolderów.

Dla każdego zestawu testowego wykonywano pomiary obejmujące wszystkie kombinacje implementacji bibliotek DLL i liczb wątków roboczych. Testowanie rozpoczynano od pojedynczego wątka, gdzie wszystkie obrazy były przetwarzane sekwencyjnie. Następnie zwiększano liczbę wątków do 2, 4, 8, 16, 32 i 64.

Testowanie z nieparzystymi liczbami obrazów było szczególnie istotne dla weryfikacji poprawności algorytmu dystrybucji zadań między wątki. Przygotowano zestawy testowe zawierające nieparzyste liczby obrazów, co przy podziale na różne liczby wątków generowało różne scenariusze podziału pracy. Weryfikacja poprawności obejmowała sprawdzenie, czy wszystkie obrazy zostały przetworzone dokładnie raz, a także sprawdzenie, czy żaden obraz nie został pominięty lub przetworzony wielokrotnie poprzez analizę zawartości folderu wynikowego.

7.3 Debugowanie

7.3.1 Debugowanie kodu w C++

Debugowanie komponentu C++ wykorzystywało zaawansowane narzędzia dostępne w środowisku Microsoft Visual Studio, w szczególności zintegrowany debugger z obsługą breakpointów, inspekcji zmiennych oraz analizy stosu wywołań. Podstawowa procedura debugowania rozpoczynała się od kompilacji projektu w trybie Debug z wyłączonymi optymalizacjami (/Od), co zapewniało pełną zgodność między wykonywanym kodem maszynowym a linijkami kodu źródłowego C++. Strategiczne umieszczanie breakpointów w kluczowych punktach programu, takich jak początek funkcji *konwertujObrazJednowatkowo(),* pętle przetwarzania wątków w *konwertujFolder()* oraz wywołania funkcji DLL, pozwalało na kontrolowane wykonywanie kodu krok po kroku i obserwację zmian stanu programu.

7.3.2 Debugowanie kodu asemblerowego

Proces debugowania rozpoczynał się od konfiguracji środowiska Visual Studio dla obsługi assemblera MASM64 poprzez dodanie pliku .asm do projektu i ustawienie typu kompilacji na "Microsoft Macro Assembler". Kompilacja w trybie Debug umożliwiała umieszczanie breakpointów bezpośrednio na instrukcjach assemblera i wykonywanie kodu krok po kroku. Podstawowa weryfikacja poprawności implementacji obejmowała sprawdzenie prawidłowego przekazywania

parametrów zgodnie z konwencją wywołań x64 Windows, gdzie pierwsze cztery parametry całkowite są przekazywane w rejestrach RCX, RDX, R8 i R9, a pozostałe parametry są umieszczane na stosie.

Kluczowym aspektem debugowania implementacji asemblerowej było monitorowanie zawartości rejestrów SIMD XMM, które przechowują wektorowe dane przetwarzane równolegle przez instrukcje SSE4.1. Visual Studio oferuje specjalne okno Registers z dedykowaną sekcją dla rejestrów XMM0-XMM15, gdzie każdy rejestr jest wyświetlany jednocześnie.

8 Podsumowanie

8.1 Wnioski dotyczące działania całego programu

Czas czystego wykonania funkcji konwersji w bibliotekach DLL stanowi znikomy ułamek całkowitego czasu działania aplikacji. Dla zestawu danych S czas wykonania DLL w wersji asemblerowej wynosił średnio 0,60538 ms przy pojedynczym wątku, podczas gdy całkowity czas działania programu osiągał 13531,8 ms, co oznacza, że konwersja stanowi zaledwie 0,0045% całkowitego czasu. Pozostałe 99,995% czasu przypada na operacje pomocnicze, w tym wczytywanie obrazów z dysku przez *cv::imread(),* konwersję przestrzeni kolorów i skalowanie obrazów przez OpenCV, renderowanie tekstowej reprezentacji ASCII do obrazu graficznego przez *cv::putText()* oraz zapisywanie wyników do plików przez *cv::imwrite()*.

Dla większych zestawów danych proporcje te zmieniają się, choć operacje wejścia-wyjścia nadal pozostają dominujące. Dla zestawu M czas wykonania DLL w wersji asemblerowej wynosił 1,7932 ms przy całkowitym czasie 52931,01 ms, co daje udział 0,0034% – niewielki wzrost względem zestawu S. Dla zestawu L czas wykonania DLL wynosił 4,4727 ms przy całkowitym czasie 92563,38 ms, co daje udział 0,0048%. Te wartości pokazują, że nawet dla dużych obrazów czas samej konwersji pozostaje marginalny wobec operacji pomocniczych.

8.2 Wnioski dotyczące wielowątkowości

8.2.1 Wielowątkowość a czas wykonania funkcji DLL

Wielowątkowość nie przyspiesza wykonania samych funkcji konwersji w bibliotekach DLL – wręcz przeciwnie, zwiększa ich czas wykonania przez wprowadzanie narzutów synchronizacyjnych i konkurencji o zasoby procesora. Dla wszystkich trzech implementacji i wszystkich zestawów danych obserwowano konsekwentny wzrost czasów wykonania DLL wraz ze zwiększaniem liczby wątków. Dla implementacji asemblerowej i zestawu S czas wykonania DLL wzrósł z 0,60538 ms przy pojedynczym wątku do 3,44922 ms przy 64 wątkach, co stanowi prawie sześciokrotny wzrost. Dla zestawu M wzrost był od 1,7932 ms do 5,7768 ms (3,2-krotnie), a dla zestawu L od 4,4727 ms do 11,27338 ms (2,5-krotnie). Charakterystyczne jest, że względny wzrost czasów maleje wraz z rozmiarem danych, co sugeruje lepszą amortyzację narzutów wielowątkowych dla większej ilości rzeczywistej pracy obliczeniowej.

8.2.2 Rzeczywiste źródła przyspieszenia

Dla folderu obrazów wielowątkowość realizuje równoległe przetwarzanie całych niezależnych obrazów – każdy wątek otrzymuje pulę kompletnych obrazów do przetworzenia i dla każdego obrazu wykonuje pełny cykl: wczytanie, konwersja, renderowanie, zapis. To jest najbardziej efektywna forma równoległości w kontekście tego projektu, gdyż różne obrazy są całkowicie niezależnymi jednostkami pracy, które nie współdzielą żadnych danych. Każdy wątek operuje na własnych lokalnych kopiach obrazów, własnych buforach pamięci i własnych wywołaniach funkcji DLL, co minimalizuje konkurencję o zasoby.

8.2.3 Optymalna liczba wątków i granice skalowalności

Granice skalowalności są wyznaczane przez kilka czynników. Po pierwsze, prawo Amdahla określa teoretyczną górną granicę przyspieszenia wynikającą z obecności sekwencyjnych fragmentów kodu, które nie mogą być zrównoleglone – w tym projekcie są to inicjalizacja bibliotek, wyświetlanie komunikatów w interfejsie oraz finalne agregowanie wyników. Po drugie, liczba fizycznych rdzeni procesora – nowoczesne procesory oferują typowo 8-16 rdzeni fizycznych, co wyznacza granicę, po której przekroczeniu wątki muszą współdzielić rdzenie fizyczne i konkurować o zasoby. Po trzecie, narzuty zarządzania wątkami przez system operacyjny – każdy wątek wymaga własnego stosu oraz czasu procesora na przełączanie kontekstu, co dla bardzo dużej liczby wątków może przeważać nad korzyściami z równoległości.

Optymalna liczba wątków powinna być dostosowana do rozmiaru przetwarzanych danych i architektury sprzętowej. Dla małych folderów niewielka liczba wątków (4-8) jest wystarczająca, gdyż dalsze zwiększanie nie przynosi znaczących korzyści. Dla średnich i dużych folderów liczba wątków zbliżona do liczby rdzeni logicznych procesora (typowo 8-16) zapewnia dobre wykorzystanie zasobów. Domyślne ustawienie liczby wątków równe *std::thread::hardware_concurrency()* (liczbie wątków logicznych procesora) stanowi rozsądny kompromis dla większości scenariuszy użytkowania.

8.3 Wnioski dotyczące różnic między wersjami bibliotek DLL

Analiza porównawcza trzech implementacji algorytmu konwersji ASCII – asemblerowej wykorzystującej instrukcje SSE4.1, C++ bez optymalizacji oraz C++ z flagą optymalizacji O2 – ujawniła różnice w wydajności czystego wykonania funkcji konwersji. Implementacja asemblerowa konsekwentnie oferowała najlepszą wydajność dla wszystkich testowanych zestawów danych. Przewaga ta wynika z precyzyjnej kontroli nad wykorzystaniem rejestrów SIMD XMM oraz ręcznej optymalizacji sekwencji instrukcji, gdzie każda operacja została świadomie zaprojektowana w celu maksymalizacji przepustowości przetwarzania wektorowego.

Implementacja C++ bez optymalizacji wykazała gorszą wydajność, osiągając czasy wykonania DLL na poziomie 8,68666 ms dla zestawu S (ponad 14-krotnie wolniej niż assembler), 26,602 ms dla zestawu M (prawie 15-krotnie wolniej) oraz 78,19258 ms dla zestawu L (ponad 17-krotnie wolniej). Ta przepaść wydajnościowa wynika z faktu, że kompilator bez flag optymalizacji generuje kod maszynowy o charakterze diagnostycznym, gdzie priorytetem jest możliwość debugowania.

Wygenerowany kod zawiera liczne sprawdzenia granic tablic, redundantne odczyty z pamięci dla tych samych zmiennych oraz brak wykorzystania równoległości na poziomie instrukcji czy rejestrów SIMD.

Implementacja C++ z optymalizacją O2 stanowi kompromis między wydajnością implementacji asemblerowej a czytelnością i łatwością utrzymania kodu wysokopoziomowego. Kompilator C++ z włączonymi optymalizacjami O2 automatycznie generuje kod wykorzystujący instrukcje SIMD oraz eliminuje redundantne obliczenia. Kluczową zaletą implementacji C++ z optymalizacją O2 jest łatwość utrzymania i modyfikacji kodu – zmiany w algorytmie wymagają jedynie edycji czytelnego kodu C++ i ponownej kompilacji.